

Projection de l'état de ressource et délégation multicritère dans une plateforme HPC à processeurs quantiques

André-Guy Bruneau, M.Sc. IT
agbruneau@gmail.com

31 août 2026 — v3 (révisée)

RÉSUMÉ

L'intégration de processeurs quantiques (QPU) dans les plateformes de calcul haute performance progresse par briques : architectures de référence, interfaces de gestion de dispositifs et de ressources, intergiciel adaptatif, stratégies de partage mesurées, plateformes déployées. Une revue critique en deux strates — quarante-deux sources ancrées affirmation par affirmation, trente-trois ajoutées par recherche systématique, dont les cinq décisives ont été relues en texte intégral — montre que la chaîne complète de délégation — décider quelle charge va sur quelle ressource, en tenant compte du fait que la qualité de sortie d'un QPU dérive dans le temps — demeure une cible que ses propres auteurs qualifient de visionnaire. Cet article isole le chaînon manquant et le spécifie. Les interfaces existantes couvrent deux niveaux disjoints — les propriétés physiques changeantes du dispositif d'un côté, l'ordonnabilité binaire de l'autre. Le mécanisme générique a un précédent classique — la grille fédérée publiait des états de ressource datés consommés par des courtiers de placement — et des fragments du lien existent — scores de compilation, ordonnancements d'accès en nuage, instantanés contractuels — mais aucune source consultée, lecture intégrale des quatre candidats les plus proches comprise, n'en publie le contenu que le QPU impose : la *projection* d'une métrique de qualité de sortie qui dérive et se périme, en un objet de première classe, daté et consommé par le gestionnaire de ressources. Nous en dérivons une architecture de référence complète, présentée en trois vues et adossée à six invariants de conception : une fonction de projection produisant un état de ressource daté, une machine d'états totale de quatre états et neuf événements, une analyse des quatre boucles de rétroaction et des quatre effets émergents qu'elles composent. Nous spécifions ensuite une politique de délégation multicritère déterministe et totale, dont la vérification n'exige ni matériel ni accès expérimental — un lecteur peut refaire le calcul à la main, et un script de rejeu versionné l'exécute —, une règle de sélection entre trois stratégies de partage d'un QPU, puis sept exigences d'exploitabilité, six procédures d'exploitation, vingt métriques de télémétrie et cinq objectifs de service, là où la littérature opérationnelle n'en publie aucun. La validation procède par matrice de couverture sur vingt exigences, rejeu de six scénarios opérationnels et confrontation à trois plateformes réelles. Sept sources ajoutées en seconde révision situent l'exploitation dans la lignée des laboratoires autopilotés et des agents scientifiques autonomes : la chaîne de délégation spécifiée ici est ce que ce consommateur non humain exigerait d'un centre à QPU, et l'article lui répond par un archétype de partie prenante, un scénario, un effet émergent et une condition de réfutation supplémentaires, sans modifier aucune contribution. Chaque affirmation tirée de la première strate a été confrontée à sa source par une passe d'ancrage dont la trace est publiée; les usages des vingt-huit sources de seconde strate non relues en intégral sont bornés à ce que leurs résumés établissent. Le travail est documentaire : la plateforme n'est pas implémentée — la politique, elle, dispose d'une implémentation de référence qui rejoue les déroulés publiés —, la structure livrée demeure numériquement non calibrée, et huit conditions de réfutation observables par un tiers sont énoncées, dont l'une a subi — et passé — son premier test lors de la révision.

Mots-clés — ingénierie des systèmes · calcul haute performance · processeur quantique · ordonnancement hétérogène · architecture de référence · exploitabilité · ISO/IEC/IEEE 15288

1 Introduction

Un centre de calcul qui ajoute un processeur quantique à son parc n'ajoute pas une ressource de plus : il ajoute une ressource dont la qualité de sortie varie dans le temps sans qu'aucune panne ne survienne. Un cœur classique délivre aujourd'hui le résultat qu'il délivrera demain; un QPU dérive entre deux étalonnages, et cette dérive est mesurée à des échelles allant de l'heure à la milliseconde [1], [2]. L'ordonnanceur d'une plateforme HPC classique n'a aucune place pour cet objet : ses ressources sont disponibles ou en panne.

Les briques de l'intégration existent pourtant et mûrissent. Des architectures de référence sont publiées [3], [4], [5], [6], [7]; des interfaces exposent le dispositif quantique — ses propriétés physiques changeantes [8] — et la ressource quantique comme ressource ordonnable de première classe [9], [10], [11]; de l'intergiciel adaptatif prédit des configurations de découpage de circuits [12]; trois stratégies de partage d'un QPU ont été comparées expérimentalement sur des grappes de production [13]; la multi-programmation spatiale d'un même dispositif est mesurée depuis 2019 [14], [15]; plusieurs plateformes sont opérationnelles [16], [17], [18], [19], [20], [21], [22], et la question de l'intégration elle-même a plus de dix ans [23]. Ce qui manque n'est donc pas une brique : c'est la chaîne entre elles. Le cadre qui décrit cette chaîne de bout en bout est qualifié de *visionnaire* par ses auteurs [5], et aucune plateforme recensée ne la réalise.

Un déplacement en cours dans la pratique scientifique donne à ce chaînon manquant son consommateur le plus exigeant. Les laboratoires autopilotés bouclent déjà sélection, exécution et interprétation d'expériences sans intervention humaine [24]; un agent fondé sur un grand modèle de langage a conçu, planifié et exécuté des optimisations de réactions réelles [25]; une plateforme autonome a opéré dix-sept jours en continu en boucle d'apprentissage actif [26]; des systèmes de bout en bout étendent la boucle jusqu'à l'hypothèse et au manuscrit [27]. Un chercheur humain qui soumet quelques charges par jour peut compenser à la main une chaîne de délégation absente — choisir sa ressource, surveiller l'étalonnage, relancer. Un agent qui enchaîne des centaines de campagnes ne le peut pas : pour lui, tout ce qui n'est pas publié comme état consommable n'existe pas. La chaîne que ce travail spécifie cesse alors d'être un confort d'exploitation; elle devient la condition à laquelle un centre à QPU peut servir la science autonome — le § 2.8 établit ce constat, et le borne.

Trois questions structurent ce travail.

- Q1** Quelle architecture de référence permet d'intégrer des processeurs quantiques en traitant leur instabilité — étalonnage périodique, dérive de fidélité — comme un état de conception et non comme une panne?
- Q2** Quels critères et quel mécanisme permettent de déléguer une charge à la ressource appropriée, et comment cette appréciation de l'adéquation se définit-elle?
- Q3** Quelles exigences d'exploitabilité distinguent une telle plateforme d'une plateforme HPC classique?

1.1 Contributions

L'article défend quatre contributions, formulées de manière à pouvoir être contestées.

1. **La projection de l'état de ressource** (§ 6). Les interfaces existantes couvrent deux niveaux : les propriétés changeantes du dispositif [8] d'un côté, l'ordonnabilité [9], [10] de l'autre. Le mécanisme générique — un état de ressource publié, daté, consommé par une politique de placement — a un précédent dans le canon distribué classique (§ 2.4); des fragments du lien existent dans la couche de compilation et dans l'ordonnement d'accès en nuage (§ 2.6), et la lecture intégrale des quatre candidats les plus proches, conduite en révision, confirme qu'aucune source des deux strates n'en publie le contenu que l'ordonnement d'un centre à QPU exige — une métrique de qualité de sortie datée, portée par une machine d'états totale dont la péremption est un événement, consommée par le gestionnaire de ressources. Nous spécifions la fonction qui projette les premières en un état daté que le second consomme, la machine d'états totale qui le porte, et l'analyse systémique des boucles que cette projection ferme.
2. **Une politique de délégation déterministe et totale** (§ 7). Pour toute entrée admissible, la politique produit une décision unique — le départage se termine sur l'identifiant de ressource, aucune configuration

ne la laisse sans réponse. Sa vérification ne demande ni matériel ni accès expérimental, et un script de jeu versionné l'exécute (§ 7.5).

3. **Une règle de sélection entre stratégies de partage** (§ 7.6). Les trois stratégies mesurées par [13] ont des domaines de validité chiffrés, mais aucune règle publiée ne choisit entre elles pour une charge donnée. Nous en proposons une, fondée sur le seul discriminant que les mesures fournissent, et dont la portée est déclarée : la multi-programmation spatiale [14], [15] est une quatrième famille de partage, qu'elle écarte explicitement (§ 7.6).
4. **Sept exigences d'exploitabilité, six procédures, vingt métriques et cinq objectifs de service** (§ 8). La littérature opérationnelle n'en publie aucun pour une plateforme HPC-QPU intégrée; les précédents partiels — DevOps quantique [28], observabilité multinuage [29], métriques mesurées d'une plateforme sans serveur [30] — sont nommés et écartés au § 8.4.

Ces contributions portent leurs limites avec elles, et le § 10 les énonce sans les adoucir. La plus lourde n'est pas technique : le travail a été conduit sans partenaire, sans contradicteur externe et sans accès expérimental.

1.2 Plan

Le § 2 établit l'état de l'art, en tire sept verrous et situe le consommateur autonome de la chaîne (§ 2.8). Le § 3 expose la méthode et ce qu'elle ne peut pas produire. Le § 4 reconstruit l'objet d'étude depuis ses contraintes et en dérive six invariants. Le § 5 définit le problème : frontière, parties prenantes, scénarios, exigences. Le § 6 présente l'architecture en trois vues. Le § 7 spécifie la politique de délégation. Le § 8 traite l'exploitabilité. Le § 9 valide. Le § 10 énonce les menaces et les conditions de réfutation.

2 Travaux connexes et cartographie des verrous

2.1 Méthode de la revue

La revue porte sur quatre-vingt-cinq sources : soixante-quinze en deux strates déclarées, trois ajoutées en révision (§ 2.4), et sept ajoutées en seconde révision en une troisième strate déclarée (§ 2.8). La **première strate** — quarante-deux sources, dont trente-quatre primaires, sept secondaires et une tertiaire, revues à la date du 29 août 2026 — constitue le corpus d'origine : chaque affirmation qui en est tirée a été confrontée à sa source par la passe d'ancrage du § 3.2. Trente-quatre de ces quarante-deux sources sont citées dans cet article; les huit autres n'interviennent que dans le mémoire dont il est tiré et n'y fondent aucune affirmation. La **seconde strate** — trente-trois sources identifiées le même jour par cinq requêtes systématiques sur un moteur de recherche académique agréant Semantic Scholar, Scopus, PubMed et arXiv — était, à la première version, ancrée **sur résumé seulement**. En révision, cinq de ses sources — les quatre quasi-antériorités du § 2.6 et l'étude de réutilisation de transpilation [31] — ont été relues en texte intégral; pour les vingt-huit autres, chaque usage reste borné à ce que le résumé établit, chaque entrée du fichier de références versionné porte la mention de ce statut, et le § 10.1 porte cette asymétrie résiduelle comme menace (M9). Enfin, la révision ajoute trois sources : deux du canon du calcul distribué classique — de statut **spécifié**, ancrées sur leurs spécifications publiques, et le § 2.4 dit pourquoi elles manquaient — et une de la littérature de décision multicritère, mobilisée au § 7.1. Le § 2.6 recense ce que la seconde strate ajoute, et ce qu'elle menace. La **troisième strate** — sept sources identifiées le 31 août 2026 par recherche systématique sur le même moteur — situe l'exploitation dans la littérature de la science autonome (§ 2.8); son ancrage est sur résumé, chaque entrée du fichier de références en porte la mention, aucune affirmation de performance n'en est tirée, et le § 10.1 la borne comme menace (M10).

Chaque source est classée à la lecture selon deux axes qui commandent l'usage qu'on peut en faire : **nature** — primaire, secondaire, tertiaire — et **statut de preuve** — mesuré, spécifié, annoncé. Une architecture de référence publiée est une spécification, pas une mesure; un communiqué de fournisseur est une annonce. La règle qui en découle est tenue dans tout l'article : aucune affirmation de performance ne s'appuie sur une annonce.

Le domaine impose une contrainte que la revue assume plutôt qu'elle ne la masque : une part importante de la littérature pertinente est constituée de préimpressions récentes [11], [19], [20], [32], [33], [34], [35], dont certaines seront révisées ou infirmées. Les écarter aurait produit un état de l'art de deux ans de retard sur un domaine qui bouge en mois.

2.2 Architectures de référence

Quatre architectures de référence sont recensées. Elles partagent une structure en couches et une préoccupation d'hétérogénéité des fournisseurs.

Architecture	Ce qu'elle apporte	Ce qu'elle ne traite pas
<i>Quantum-centric supercomputing</i> [3]	Architecture de référence complète, centrée sur le QPU comme raison d'être du système; intégration au flux de travail scientifique	L'état de la ressource reste binaire du point de vue de l'ordonnancement
openQSE [4]	Survey et architecture de référence de pile logicielle; recense quatre besoins communs des piles en production, dont l'observabilité	Nomme le besoin d'observabilité sans en dériver un état ordonnable
Cadre QHPC [5]	Décrit la chaîne complète, de la soumission unifiée à l'exécution; interface de soumission indépendante du type de calcul et de sa localisation	Qualifié de <i>visionnaire</i> par ses propres auteurs; aucune réalisation
Cadre agnostique ORNL [6], [7]	Conception pour la convergence HPC-quantique; agnosticisme vis-à-vis du fournisseur	L'instabilité de la ressource n'est pas un état de première classe propagé

Tableau 1 — Les quatre architectures de référence recensées.

Ce qu'aucune ne traite. Les quatre traitent l'hétérogénéité des fournisseurs — comment brancher plusieurs technologies de QPU derrière une même façade. Aucune ne fait de l'état de la ressource un état de première classe propagé jusqu'à l'ordonnanceur. C'est le premier verrou.

2.3 Interfaces de gestion et intergiciel

Deux interfaces normalisées émergent, et elles ne couvrent pas le même niveau.

QDMI [8] décrit la soumission et le contrôle de systèmes à portes, et la récupération automatique des propriétés physiques et contraintes changeantes des plateformes, valeurs de fidélité vivantes comprises; l'interface est fournie en en-têtes C. C'est le niveau du dispositif.

QRMI [9], [10] expose la ressource quantique comme ressource ordonnançable de première classe, par une API unifiée, une intégration via les mécanismes natifs des gestionnaires de charge, et un cycle de vie acquisition-exécution-libération, validé sur six environnements. C'est le niveau de l'ordonnancement.

Du côté des gestionnaires de charge, l'intégration progresse par le mécanisme des ressources génériques : exposition des QPU sans modification du cœur de l'ordonnanceur [19], extensions des systèmes de gestion de ressources [11], ordonnancement par graphe [32], et intégrations industrielles annoncées [36], [37] — ces deux dernières étant des annonces, dont aucune affirmation de performance n'est tirée ici. L'annonce NVQLink est depuis adossée à une publication d'architecture [38] (seconde strate), dont le résumé rapporte une latence aller-retour mesurée de 3,96 μ s : le statut d'annonce ne vaut plus que pour [36].

Enfin, de l'intergiciel adaptatif gère ressources, charges et tâches, et prédit les configurations optimales de découpage de circuits avec **jusqu'à 82 % de précision** sur plateformes hétérogènes [12]. Ce chiffre est le plus utile de la revue, non parce qu'il est élevé, mais parce qu'il est **inférieur à 100 %** : il rend chiffrable l'incertitude qu'une politique doit traiter. Aucune source ne spécifie le comportement attendu dans le complément.

2.4 Ordonnancement hybride : héritage et ruptures

L'ordonnancement hétérogène classique fournit l'héritage : files, priorités, quotas, réservations.

L'héritage ne se limite pas au centre de calcul consolidé — et la première version de cet article l'avait manqué. L'informatique de grille a affronté, dès la fin des années 1990, deux des configurations que ce travail

traite : la ressource fédérée que l’ordonnanceur ne possède pas, et l’état de ressource publié comme objet de première classe. Le *matchmaking* de HTCondor apparie charges et ressources sur des ClassAds — des descripteurs d’attributs publiés par les ressources elles-mêmes, rafraîchis périodiquement, consommés par un appariteur central, conçus précisément pour la propriété distribuée et les politiques d’allocation hétérogènes [39]. Le schéma GLUE normalise l’information de ressource des infrastructures de grille — entités, attributs, fraîcheur — consommée par les courtiers de placement [40]. La non-coïncidence entre frontière de propriété et frontière du système (§ 5.1) n’est donc pas inédite, et le mécanisme générique de la projection du § 6.3 — un état publié, daté, consommé par une politique — a un précédent classique que cette lignée établit. Ce que ce canon ne contient pas, et que la revue n’a trouvé nulle part ailleurs, c’est le **contenu** que le QPU impose à cet état : une métrique de qualité de sortie qui dérive entre étalonnages, portée par un cycle de vie où la péremption est un événement et l’étalonnage un état nominal. Un ClassAd peut porter n’importe quel attribut; aucune source consultée n’en publie un qui porte une fidélité datée consommée comme critère de placement. La contribution du § 6.3 se positionne donc ainsi : le mécanisme a un précédent, le contenu et la sémantique de cycle de vie n’en ont pas.

Trois hypothèses de cet héritage se rompent en présence d’un QPU.

1. **La qualité de sortie d’une ressource ne varie pas.** Elle varie ici, et à des échelles mesurées de l’heure [1] à la milliseconde [2]. Cette rupture est propre au QPU : ni le centre consolidé ni la grille ne l’ont connue.
2. **L’ordonnanceur possède ses files.** Il n’en possède qu’une part : la file du fournisseur est une seconde file, et elle est opaque [32], [41]. Rupture pour le centre consolidé; quotidien de la grille fédérée, dont les mécanismes de description d’état [39], [40] n’ont toutefois jamais porté de métrique de qualité qui se périme.
3. **L’indisponibilité est une exception.** Elle est ici périodique et, dans le meilleur cas, annoncée.

Sur le partage d’un QPU entre charges, trois stratégies ont été comparées expérimentalement sur des grappes HPC de production et du matériel réel [13] : multiplexage temporel, gestion dynamique des ressources (malléabilité) et décomposition du flux de travail. Les résultats donnent des domaines de validité chiffrés — réduction de la consommation de ressources classiques jusqu’à 45,7 % et 64 % pour les deux dernières, meilleure utilisation du QPU pour la première en régime déséquilibré. Aucune règle publiée ne choisit entre elles pour une charge donnée.

2.5 Plateformes opérationnelles

Six plateformes au moins exploitent un QPU aux côtés de ressources classiques [16], [17], [19], [20], [21], [22], selon des topologies qui vont du couplage de type service au couplage serré. Trois d’entre elles servent de banc de confrontation au § 9.3.

Le constat décisif est **négatif et il est solide** : aucune de ces plateformes ne publie de taux de disponibilité, de périodicité d’étalonnage ni de distribution d’attente. La seule mesure de disponibilité de la base — 92 % sur six mois — porte sur un service en nuage et non sur une plateforme intégrée [42]. Elle fixe un plafond de crédibilité, non une cible transposable.

2.6 Seconde strate : quasi-antériorités et compléments

Cinq requêtes — architecture d’intégration, ordonnancement hétérogène, dérive d’étalonnage, partage d’un QPU, exploitabilité — ont ajouté trente-trois sources au corpus le 29 août 2026, sous le statut déclaré au § 2.1 : ancrage sur résumé, usages bornés en conséquence. Quatre apports en sortent, et le premier menace directement la revendication centrale — c’est la raison d’être de cette section.

Quatre quasi-antériorités de la projection. Quatre sources publient chacune un fragment de ce que le § 6.3 spécifie. À la première version, leur écartement reposait sur les résumés; la révision a conduit la passe d’ancrage intégrale des quatre — la colonne de droite en porte le résultat.

Source	Ce que le résumé établit	Ce qui la sépare de la projection — confirmé en texte intégral
HPC-vQPU [43]	Instantané de dispositif sensible à la topologie et à l'étalonnage, lié atomiquement au moment de la réclamation et porté dans l'exécution comme contrat immuable; empêche l'exécution sur un état périmé	La lecture intégrale confirme : la liaison à la réclamation (<i>claim-time binding</i>) prévient l'exécution sur état périmé, mais l'instantané est un contrat immuable — ni machine d'états, ni cycle de qualité, ni consommation par une politique de placement — et l'objet virtualisé est un simulateur (Qiskit-Aer/cuQuantum) adossé à des données d'étalonnage, non un dispositif qui dérive. Candidat le plus proche de RÉF-2; ne la déclenche pas
Qurator [44]	Ordonnancement conjoint temps de file $\times$ fidélité entre fournisseurs d'accès en nuage; score de succès unifié réconciliant les données d'étalonnage de six fournisseurs	La lecture intégrale confirme : le coefficient d'ordonnancement est calculé au moment du placement et consommé par le seul ordonnanceur de Qurator; il n'est ni publié comme état daté, ni porté par un cycle de vie, et la péremption n'y est pas un événement
Mapomatic [45]	Fonctions de coût dérivées des données d'étalonnage pour noter des sous-graphes candidats; récupère en moyenne près de 40 % de la fidélité perdue au placement de circuits	La lecture intégrale confirme : passe de transpilation (par défaut depuis Qiskit 0.21), score recalculé par circuit, données d'étalonnage « nominalement mises à jour quotidiennement » sans traitement de leur péremption; rien n'est publié vers un ordonnanceur. La lignée remonte à la compilation adaptative au bruit [46]
Qonscious [47]	Exécution conditionnelle de programmes quantiques fondée sur une évaluation dynamique des ressources	La lecture intégrale confirme — et renforce : le cadre vit côté application (<code>run_conditionally()</code>), sans intégration à un gestionnaire de ressources, et nomme lui-même le découplage temporel entre introspection et exécution comme défi ouvert — précisément ce que la projection traite

Tableau 2 — Quatre quasi-antériorités de la projection, et ce qui les en sépare. Les quatre ont été relues en texte intégral en révision; la séparation décrite a résisté à la lecture (§ 10.1, § 10.2).

Le constat de la première strate — « aucune source ne publie le lien » — doit donc être resserré. Des fragments de la projection existent : dans la couche de compilation, dans l'ordonnancement d'accès en nuage, dans la virtualisation de dispositifs. Ce qu'aucune source des deux strates ne publie — et la passe d'ancrage intégrale des quatre candidats les plus proches le confirme —, c'est la projection comme **objet de première classe du gestionnaire de ressources d'un centre** : un état daté, porté par une machine d'états totale dont la péremption est un événement, consommé par une politique de délégation publiée. Le § 6.3 défend cette forme resserrée; RÉF-2 (§ 10.2) reste la condition qui la ferait tomber, et elle a résisté à son premier test.

La prémisse de fraîcheur, appuyée et contestée. La dérive et ses échelles sont établies par la première strate [1], [2]. La seconde ajoute les deux faces d'une tension que l'article convertit en condition de réfutation plutôt que de la trancher. D'un côté : les données d'étalonnage publiées par les fournisseurs deviennent obsolètes en quelques minutes, et le prétraitement d'historique améliore la fidélité quand l'étalonnage en temps réel n'est pas disponible [48]; la quantification d'incertitude sur plusieurs jours capture la dérive et son coût de maintenance [49]; l'étalonnage en boucle fermée piloté en temps réel est démontré [50]; le panorama de référence des métriques d'étalonnage-mesure existe [51], et un outillage interactif agrège déjà l'étalonnage en scores de disposition datés, à quatre modes temporels [52]. De l'autre : une étude sur six machines de 127 qubits et seize algorithmes conclut que des circuits compilés une fois se réutilisent de façon fiable sur plusieurs cycles d'étalonnage, et que la compilation sensible au bruit concentre la charge sur un petit sous-ensemble de qubits au point d'accroître la variabilité des sorties [31]. Sa lecture intégrale, conduite en révision, en borne la portée — et a corrigé au passage le nombre de machines, que la première version donnait à cinq : l'étude mesure la réutilisation d'une transpilation **sur une même machine** au fil des cycles; elle ne traite ni du choix entre dispositifs ni de la fraîcheur comme critère de sélection. Si la fraîcheur

importe moins que la prémisse ne le suppose, la valeur de la projection baisse d'autant : c'est la condition RÉF-7 (§ 10.2), que cette source rend plausible sans pouvoir la déclencher.

La multi-programmation spatiale, quatrième famille de partage. Le § 2.4 recense trois stratégies temporelles mesurées [13]. Une quatrième famille, spatiale, est mesurée depuis 2019 : partitionnement d'un dispositif en régions fiables construites sur l'étalonnage, avec bascule adaptative vers le mode mono-programme [14]; gestion du parallélisme et caractérisation de la diaphonie [15]; partitionnement par détection de communautés [53]; allocation dynamique atteignant 92 % d'utilisation moyenne [54]; formulation exacte en nombres entiers, NP-difficulté et heuristiques [55]. La règle du § 7.6 déclare cette famille hors de sa portée, et dit pourquoi.

Gestion de ressources, orchestration, exploitation. Le paysage du § 2.3 s'élargit sans changer de forme : malléabilité des allocations classiques pendant le déport quantique [56], tâches hétérogènes Slurm [57], orchestration infonuagique native [58], placement multi-locataire sensible au réseau [59], ordonnancement de circuits sur QPU non identiques formulé en programme linéaire [60], taxonomie et simulateur des approches de gestion de charge [61], et un contre-modèle appris — estimation de ressources par réseau de graphes, choix du mode d'exécution par apprentissage par renforcement [62] — qui est l'exact opposé de la politique déterministe du § 7. Côté architecture, la question de l'intégration a plus de dix ans — deux voies, serrée et lâche, dès 2015 [23] —, les panoramas récents confirment la fragmentation [63], une architecture multiniveau portant une couche d'étalonnage explicite est déployée dans deux centres européens [64], et une couche d'abstraction quantique au niveau du noyau est proposée [65]. Côté exploitation, trois précédents partiels [28], [29], [30] sont examinés et écartés au § 8.4. Enfin, deux sources documentent une menace que la première strate n'avait pas vue — la falsification de l'entrée d'étalonnage [66] et l'injection adverse en multi-tenance [67] — traitée au § 8.6.

2.7 Sept verrous

Verrou	Nature	Constat qui l'établit	Question
V1 — l'instabilité de la ressource n'est portée par aucune vue architecturale	objet	Les quatre architectures du § 2.2 traitent l'hétérogénéité des fournisseurs; aucune ne fait de l'état de ressource un état de première classe propagé jusqu'à l'ordonnanceur [3], [4], [5], [6]	Q1
V2 — le cycle de vie des interfaces encode une disponibilité binaire	objet	Le cycle acquisition-exécution-libération de QRMI [9], [10] n'a pas d'état pour « acquise mais dégradée »; QDMI expose les propriétés changeantes [8] sans que la couche d'ordonnancement en dérive un état	Q1
V3 — la chaîne complète de délégation n'existe qu'à l'état de cible	objet	Le cadre qui la décrit de bout en bout est qualifié de visionnaire par ses auteurs [5]; aucune plateforme recensée ne la réalise	Q2
V4 — les modèles prédictifs sont partiels et leur zone d'échec n'est pas spécifiée	lien	Précision jusqu'à 82 % sur le découpage de circuits [12]; aucune source ne spécifie le comportement de la politique dans le complément	Q2
V5 — la sélection entre stratégies de partage n'est pas spécifiée	lien	Trois stratégies aux domaines de validité mesurés [13]; aucune règle publiée ne choisit entre elles pour une charge donnée	Q2
V6 — aucune donnée d'exploitation n'est publiée pour une plateforme HPC-QPU intégrée	preuve	Six plateformes déployées; zéro taux de disponibilité, zéro périodicité d'étalonnage, zéro distribution d'attente publiés. La seule mesure disponible [42] porte sur un service en nuage	Q3

V7 — mesures de dérive et procédures d'exploitation ne sont pas reliées	lien	Dérive mesurée à deux échelles [1], [2], rééchantillonnage déclaré pilotable [68], observabilité spécifiée [34] — aucune source ne relie une mesure de dérive à une procédure ni à un objectif de service	Q3
---	------	---	----

Tableau 3 — Sept verrous issus de la revue critique. La colonne « nature » distingue ce qui manque : un **objet** (rien n'existe), une **preuve** (l'objet existe, sa démonstration non), un **lien** (deux objets existent, leur articulation non).

Trois de ces verrous sont des liens manquants (V4, V5, V7), deux des objets manquants au niveau du modèle (V1, V2), un un objet manquant au niveau de la chaîne (V3), un une absence de preuve publiée (V6). Cette répartition délimite les prétentions de l'article : V1 à V5 et V7 sont attaquables par un travail de conception documentaire; **V6 ne l'est pas**. Aucun travail documentaire ne comble une absence de données publiées, et le § 10 le porte comme limite plutôt que comme perspective.

Un point mérite d'être relevé pour ce qu'il dit du domaine : sur les sept verrous, aucun ne porte sur le matériel, un seul sur la disponibilité de la preuve, et six sur la conception et l'articulation des couches logicielles et opérationnelles.

La seconde strate (§ 2.6) ne referme aucun des sept verrous, mais elle en déplace deux — et elle conteste une prémisse. Elle fournit à V2 son candidat de réfutation le plus proche, l'instantané contractuel de [43] — dont la lecture intégrale, conduite en révision, a confirmé qu'il ne le referme pas. Elle borne V1 : une architecture multiniveau déployée porte une couche d'échantillonnage explicite [64], sans que son résumé ni la page de l'éditeur — seuls éléments accessibles à la révision — permettent d'établir si l'ordonnanceur y consomme un état : le texte intégral devra trancher. Elle conteste enfin ce que la première strate n'interrogeait pas : la valeur de la fraîcheur elle-même [31], convertie en condition de réfutation RÉF-7 (§ 10.2) et bornée depuis par la lecture intégrale de sa source (§ 2.6).

2.8 Troisième strate : le scientifique autonome comme consommateur

La seconde révision ajoute une strate courte — sept sources, identifiées le 31 août 2026, ancrées sur résumé — dont l'objet n'est pas un verrou de plus : elle établit **pour qui** la chaîne de délégation manquante compte le plus. Elle ne modifie aucune contribution; elle situe l'exploitation.

Le fait que cette littérature établit est une bascule du consommateur. Un laboratoire autopiloté est une plateforme modulaire qui choisit, exécute et interprète itérativement ses propres expériences pour atteindre un objectif déclaré [24]. La démonstration est mesurée : un agent fondé sur un grand modèle de langage a conçu, planifié et exécuté des optimisations de réactions réelles [25]; une plateforme autonome a opéré dix-sept jours en continu, en boucle d'apprentissage actif, sur cinquante-huit cibles de synthèse [26] — et la portée de ses revendications de nouveauté a été corrigée par la revue après contestation publique [69], rappel utile de la jeunesse de cette littérature. L'accès distant existe — des expériences dirigées par IA s'exécutent dans des laboratoires en nuage accessibles par abonnement [70], la configuration même de la topologie T1 du § 6.2 —, et des systèmes de bout en bout étendent la boucle jusqu'à la génération d'hypothèses et au manuscrit [27]. La littérature de synthèse qui organise ce paysage nomme ses verrous — auditabilité, reproductibilité, interopérabilité de l'exécution — et met en garde contre la confusion entre agent outillé et découverte autonome [71].

Deux constats en sortent pour ce travail, et ils tirent dans des directions différentes.

Le premier justifie. Aucune source de cette strate n'opère un instrument dont la qualité de sortie dérive et se périmé entre échantillonnages : le robot de synthèse, le spectromètre, la ligne d'écoulement y sont supposés stables, leur disponibilité binaire. Un QPU n'offre ni l'un ni l'autre (INV-2). Un agent scientifique qui prend un centre à QPU pour instrument a donc besoin de ce que cette littérature ne fournit pas et que cet article spécifie : un état de ressource daté et publié (§ 6.3), une décision motivée dont les alternatives et la confiance sont des sorties de première classe (§ 7.2), des procédures dont l'état est reconstituable sans opérateur humain (§ 8.5). Les verrous que la littérature agentique nomme — auditabilité, reproductibilité [71] — sont précisément ce que ces objets réalisent.

Le second borne. Cette strate est la plus jeune du corpus — une préimpression [27], une correction publiée après contestation [69] —, son ancrage est sur résumé, et aucune affirmation de performance n'en est tirée. Son usage est confiné à quatre endroits déclarés : la motivation (§ 1), un archétype de partie prenante (§ 5.2), un scénario opérationnel (§ 5.3) et un effet émergent (§ 6.5). Le § 10.1 la porte comme menace (M10), et le § 10.2 lui associe la condition de réfutation qui la viderait (RÉF-8).

3 Méthode

3.1 Cadre

La démarche déroule les processus techniques du cycle de vie des systèmes, de l'analyse de mission à la validation : besoins et exigences des parties prenantes, définition de l'architecture, conception, exploitation et maintenance, vérification et validation. La structure s'inspire des processus techniques d'ISO/IEC/IEEE 15288:2023 [72] — **inspire** est le mot juste, et le paragraphe suivant dit pourquoi il ne peut pas être plus fort. Trente processus ont été passés en revue, seize retenus et quatorze écartés avec justification — un usage du cadre comme **grille de complétude**, non comme référentiel de conformité.

Une déclaration s'impose ici, et elle conditionne la lecture du reste. Le texte normatif n'a pas été acquis — barrière payante — et la structure de la norme n'est mobilisée qu'au travers d'une restitution tertiaire [73]. Le cadre est donc déclaré **inspiré de** la norme, non adossé à elle : **aucune revendication de conformité n'est faite**, la grille de complétude elle-même repose sur une restitution de seconde main, et sur les points où la norme est précise, cet article ne peut rien affirmer. Acquérir le texte, ou abandonner jusqu'à la référence d'inspiration, sont les deux issues honnêtes; la seconde perdrait la seule grille publique qui couvre le cycle complet, et c'est ce qui justifie l'entre-deux déclaré ici.

Le cadre classique capture par ailleurs mal quatre traits du problème, et il vaut mieux les nommer que les subir. L'exigence y est supposée satisfaite ou non, alors qu'ici elle l'est **par intermittence**. La vérification y est supposée répétable, alors que la mesure sur QPU ne l'est pas. La frontière du système y est supposée coïncider avec la maîtrise, alors qu'une part de l'état est opaque. Exploitation et maintenance y sont séparées, alors que l'étalonnage relève des deux. Chacun de ces écarts a une conséquence traçable dans la suite : le verdict « couverte sous réserve » du § 9.1, la stratégie de vérification documentaire du § 9, l'invariant INV-5 du § 4, et le traitement de l'étalonnage comme état nominal du § 8.

3.2 Passe d'ancrage

Chaque affirmation factuelle a été confrontée à sa source dans une passe systématique, dont la trace est publiée par chapitre du mémoire dont cet article est tiré, sous forme de trois listes : confirmés, dégradés en hypothèse déclarée, retirés. Sur les neuf chapitres factuels, **deux cent dix-huit affirmations ont été examinées, trente-sept dégradées et vingt-neuf retirées**.

La passe couvre la première strate du corpus. En révision, elle a été étendue à cinq sources de la seconde strate — les quatre quasi-antériorités du § 2.6 et [31] — relues en texte intégral : une valeur en est sortie corrigée (six machines, non cinq, dans [31]), aucun usage n'a dû être retiré, et l'écartement des quasi-antériorités a été confirmé. Les vingt-huit autres sources de seconde strate n'y sont pas soumises : leurs usages sont bornés au résumé de chaque source, chaque entrée du fichier de références versionné en porte la mention, et le § 10.1 enregistre cette asymétrie résiduelle comme menace M9.

Deux conventions typographiques en découlent, tenues dans tout l'article. Une valeur numérique non issue d'une source publiée porte la mention explicite *hypothèse de travail*. Une déduction de l'auteur que les sources n'énoncent pas est introduite comme telle — *inférence de l'auteur* — et jamais présentée sur le ton d'un constat.

3.3 Ce que la méthode ne peut pas produire

Trois choses, et il vaut mieux les nommer ici qu'en conclusion. Elle ne produit **aucune mesure** : ni temps de complétion, ni taux de placement, ni disponibilité observée. Elle ne produit **aucune calibration** : les poids, seuils et bornes de la politique du § 7 sont des hypothèses de travail. Elle ne produit **aucune validation**

externe : les sept parties prenantes du § 5.2 sont des archétypes inférés de traces écrites, et un besoin qu’aucun document ne mentionne parce qu’il va de soi pour qui exerce le métier est structurellement invisible à cette approche.

4 Fondements : six invariants de conception

Le modèle de la ressource est reconstruit depuis ses contraintes plutôt que par analogie avec une ressource classique. Six invariants en sont dérivés; chacun est accompagné, dans le mémoire dont cet article est tiré, du contre-exemple qui le réfuterait — un invariant sans contre-exemple énonçable n’est pas un invariant, c’est une opinion.

Invariant	Énoncé	Ce qui le réfuterait
INV-1	Le couplage n’est utile qu’à proportion du poids de la communication dans le temps total.	Une mesure montrant un gain de co-localisation sur une charge dont la communication est négligeable dans le temps total.
INV-2	Une ressource à qualité dérivante n’est pas descriptible par sa seule disponibilité.	Une politique de placement correcte n’employant que l’état binaire, sur une ressource dont la fidélité dérive.
INV-3	Une indisponibilité périodique connue à l’avance est une contrainte d’ordonnancement, non une exception.	Une publication établissant qu’un rééquilibrage s’exécute pendant qu’une charge utilisateur occupe le dispositif.
INV-4	Une politique définie sur des signaux post-exécution n’est pas exécutable.	Un estimateur pré-exécution restituant la durée ou la fidélité effectives avec une erreur bornée et publiée.
INV-5	Une part de l’état d’une ressource tierce est inobservable, et la décision doit s’en passer.	Un fournisseur publiant la profondeur instantanée de sa file propre.
INV-6	La rareté déplace le critère de décision vers le coût d’opportunité.	Un parc où placer une charge n’en exclut aucune autre — c’est-à-dire une ressource abondante.

Tableau 4 — Six invariants de conception, chacun avec la condition qui le réfuterait.

INV-2 est celui dont le reste découle. Si la disponibilité ne suffit pas à décrire la ressource, alors l’ordonnanceur a besoin d’un objet que ni QDMI ni QRMI ne lui fournit — et c’est exactement la contribution du § 6.3. **INV-4 est celui qui contraint le plus la politique** : il exclut de la table des entrées la durée effective, la fidélité obtenue et le nombre d’itérations avant convergence d’une campagne variationnelle, qui sont précisément les trois grandeurs qu’on aimerait connaître. **INV-5 est celui qui coûte le plus cher** : il impose de spécifier ce que la décision fait en l’absence d’information, plutôt que de supposer l’information disponible.

5 Définition du problème

5.1 Frontière du système et environnement

Le système est la plateforme de calcul consolidée : la couche d’accès, l’orchestration, l’ordonnanceur et sa politique, la couche d’abstraction matérielle, les ressources de calcul et la télémetrie. Cinq entités restent **hors frontière**, et cette exclusion est ce qui rend le problème difficile plutôt qu’un détail de cadrage.

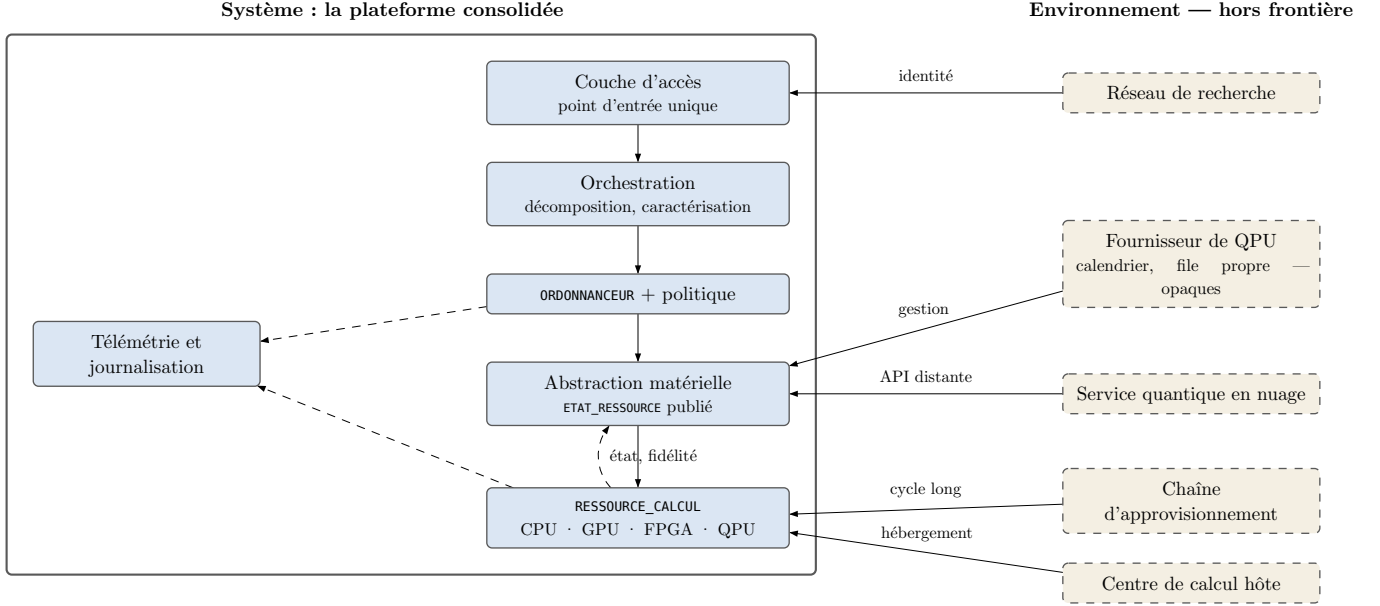


Fig. 1 — Frontière du système et environnement. Les cinq entités en trait discontinu sont hors frontière : la plateforme subit leurs décisions sans les prendre. Le fournisseur de QPU en particulier détient le calendrier d’étalonnage et une file propre, tous deux opaques (INV-5).

Le cas déterminant est le fournisseur de la ressource quantique. Il décide de l’étalonnage et exploite sa propre file; la plateforme ne peut ni l’un ni l’autre. C’est une **non-coïncidence entre la frontière de propriété et la frontière du système** — configuration que le centre HPC consolidé ne connaît pas dans son plan d’ordonnancement, mais que la grille fédérée a affrontée avant lui (§ 2.4) : INV-5 en est la traduction en contrainte de conception, et il s’inscrit dans cette lignée. Ce qui n’a pas de précédent, c’est que la part opaque porte ici une qualité de sortie qui dérive, non une simple charge.

Le budget de latence de l’ordre de la microseconde qu’exigent les boucles de correction d’erreurs [33] est également hors frontière : aucune décision de plateforme ne s’y loge. Cette exclusion délimite la portée de tout ce qui suit.

5.2 Parties prenantes et tensions

Six archétypes de parties prenantes sont construits sur des traces écrites : chercheur soumissionnaire, développeur d’applications hybrides, opérateur de plateforme, administrateur d’ordonnanceur, direction du centre, fournisseur de ressource quantique. Chacun porte, dans le mémoire, le champ « ce qui manque » qui consigne ce que l’absence d’entrevue laisse ignorer.

La seconde révision ajoute un septième archétype, construit sur les traces écrites de la troisième strate (§ 2.8) : **l’agent scientifique autonome**, soumissionnaire non humain qui conduit des campagnes en boucle fermée hypothèse–expérience–analyse [24], [25]. Trois traits le distinguent du chercheur soumissionnaire. Sa cadence de soumission n’a pas de borne physiologique — la borne de consommation (règle R4) et le plancher d’équité (PR-04), pensés pour des tenants humains, deviennent ses seuls régulateurs. Il ne consomme que des interfaces machine — un état publié (I4), une décision motivée (I5), une télémétrie (I9) — de sorte que tout ce qui n’est pas publié n’existe pas pour lui : le principe du § 6, « ce que la plateforme ne maîtrise pas, elle le déclare », passe de règle de conception à condition de fonctionnement. Enfin, il ne s’arrête pas de lui-même sur une valeur invraisemblable : la falsification de l’entrée d’étalonnage (§ 8.6) atteint sa boucle de décision sans le regard humain qui pourrait tiquer. Son champ « ce qui manque » est plus lourd que celui des six autres : aucune entrevue, aucun déploiement observé sur plateforme HPC-QPU, et une littérature source qui est la plus jeune du corpus (M10).

Les besoins ne se concatènent pas : quatre tensions les traversent, et les traiter comme une liste masquerait l’arbitrage.

Tension	Termes	Arbitrage retenu
Utilisation contre équité	La direction veut maximiser l’occupation d’une ressource coûteuse; le chercheur veut que sa charge passe. Optimiser l’occupation seule affame les petits tenants.	Une borne de consommation est posée comme exigence, non comme réglage. L’effet de famine est nommé au § 6.5 et traité au § 8.
Transparence contre opacité	L’opérateur veut voir l’état; le fournisseur ne publie que ce qu’il consent à publier.	La plateforme ne peut pas exiger l’information. Elle exige de déclarer ce qui manque, et de spécifier la décision en son absence (INV-5).
Promesse contre variabilité	La direction doit promettre un objectif de service; la ressource dérive entre étalonnages [1], [2].	La promesse est différenciée par classe de service plutôt qu’uniforme (§ 8.4).
Simplicité d’accès contre contrôle	Le chercheur veut ne rien décider; le développeur veut parfois décider — imposer une ressource pour comparer deux exécutions.	Le point d’entrée unique est le cas nominal; la désignation explicite reste une variante déclarée, dont le coût en équité est porté au § 8.
Cadence contre équité	L’agent autonome soumet à une cadence sans borne physiologique; les tenants humains partagent la même ressource rare.	Aucun mécanisme nouveau : la borne de consommation (R4) et le plancher d’équité (PR-04) s’appliquent par tenant, sans distinguer humain et agent. Que leur calibrage suffise à une cadence machine est une hypothèse déclarée, et l’effet de résonance est porté au § 6.5.

Tableau 5 — Cinq tensions entre parties prenantes, et l’endroit où chacune est arbitrée. La cinquième arrive avec le septième archétype.

5.3 Scénarios opérationnels et exigences

Six scénarios opérationnels servent à la fois de définition du besoin et de banc de validation (§ 9.2) : soumission hybride, campagne variationnelle, étalonnage planifié, dégradation imprévue, multi-tenance sous contention, et — ajouté en seconde révision — campagne pilotée par un agent autonome, où le soumissionnaire est le septième archétype du § 5.2 et où chaque itération de la boucle hypothèse–expérience–analyse consomme l’état publié et la décision motivée. Dégradation imprévue et multi-tenance sous contention sont les scénarios de rupture, et ce sont eux qui ne passeront que partiellement.

Vingt exigences en sont dérivées — huit fonctionnelles, douze non fonctionnelles. Les huit fonctionnelles se lisent comme la chaîne de bout en bout : soumission unifiée sans désignation de ressource, décomposition en charges, caractérisation restreinte aux signaux pré-exécution, décision par politique publiée, publication de l’état de ressource, réservation des fenêtres d’étalonnage, repli à l’indisponibilité, agrégation et restitution avec incertitude. Les douze non fonctionnelles portent l’exploitabilité, l’équité, la traçabilité et la portabilité; sept d’entre elles sont propres au QPU et font l’objet du § 8.1.

Le sixième scénario a une propriété qui mérite d’être énoncée plutôt que découverte : **il ne dérive aucune exigence nouvelle**. Soumission unifiée sans désignation de ressource, décision par politique publiée, publication de l’état, restitution avec incertitude — les huit exigences fonctionnelles couvrent déjà un soumissionnaire machine, parce qu’elles ont été formulées en interfaces plutôt qu’en écrans. Il en durcit deux : l’observabilité de la décision et la publication de l’état cessent d’être des exigences de traçabilité pour devenir le canal de fonctionnement du consommateur. Le décompte de vingt exigences est donc inchangé, et la matrice du § 9.1 n’a pas de ligne nouvelle à porter.

6 Architecture de référence

L’architecture est présentée en trois vues — fonctionnelle, physique, dynamique — dont la cohérence est vérifiée élément par élément. Cinq principes la gouvernent, dont un seul mérite d’être énoncé ici parce qu’il commande les autres : **ce que la plateforme ne maîtrise pas, elle le déclare**. Une dégradation déclarée est une information; une dégradation masquée est un défaut.

6.1 Vue fonctionnelle

Cinq couches, seize fonctions, treize interfaces. Chaque fonction porte l'exigence qui la fonde : aucune fonction sans exigence source, aucune exigence fonctionnelle sans fonction.

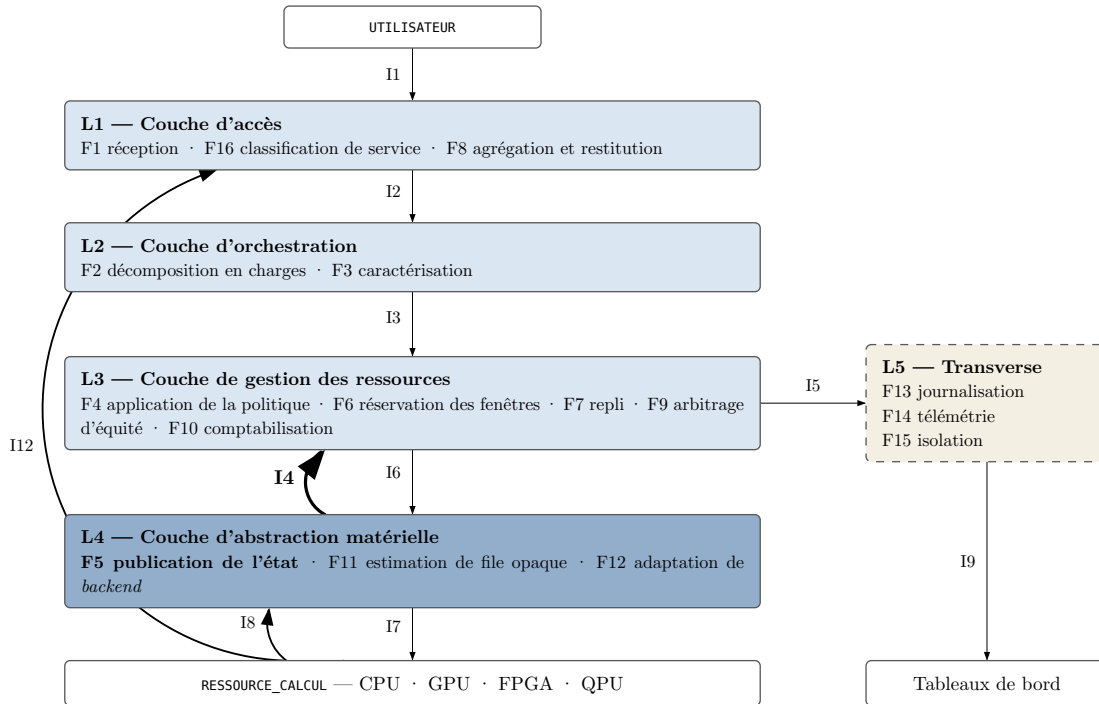


Fig. 2 — Vue fonctionnelle : cinq couches, seize fonctions et les interfaces nommées qui les relient. L'interface I4, en trait renforcé, est celle qui n'a d'équivalent publié dans aucune des architectures recensées.

Interface	Producteur → consommateur	Contenu
I1	utilisateur → F1	descripteur de soumission
I2	F2 → F3	descripteur de charge de travail et graphe de dépendances
I3	F3 → F4	caractérisation de la charge
I4	F5 → F4, F14	état de ressource et métrique de fidélité datée
I5	F4 → F7, F13	décision de délégation
I6	F7 → F12	ordre d'exécution
I7	F12 → ressource	interface de gestion : acquisition, exécution, libération
I8	ressource → F5	propriétés et contraintes du dispositif, valeurs de fidélité
I9	F14 → tableaux de bord	flux de télémétrie
I10	F6 → F4	réservations de fenêtres d'étalonnage sur l'horizon
I11	F11 → F4	estimation d'attente et son incertitude
I12	ressource → F8	résultat d'exécution
I13	F9 → F4	part consommée et priorité effective du tenant

Tableau 6 — Les treize interfaces, chacune avec son producteur et son consommateur. Cinq d'entre elles convergent vers F4 : la politique de délégation est le point de rencontre de l'architecture.

Cinq des treize interfaces alimentent F4. Ce n'est pas un accident de découpage : la fonction de décision est l'endroit où l'information hétérogène — caractérisation, état, réservations, estimations, quotas — doit se réduire à un choix unique. Le § 7 est la spécification de cette réduction.

6.2 Vue physique : topologies et degré de couplage

Trois topologies. INV-1 interdit d'en désigner une comme universellement supérieure : le gain d'une réduction de latence est borné par la part du temps de communication dans le temps total.

Topologie	Ce qu'elle donne	Ce qu'elle coûte
T1 — service distant	Accès sans investissement matériel; catalogue de technologies large	Seconde file appartenant au fournisseur [32]; file opaque [9]; attente d'accès en nuage mesurée de la minute à la journée [44]; aucune maîtrise du calendrier d'étalonnage
T2 — co-localisé	File unique; réservation de la fenêtre d'étalonnage possible; télémétrie de bout en bout	Investissement d'hébergement : étude de site, redondance d'alimentation et de refroidissement [68]
T3 — intégration serrée	Rend possibles les boucles temps réel	Budget de latence hors d'atteinte de toute décision de plateforme; portée réduite à la boucle de correction [33]

Tableau 7 — Trois topologies et leur compromis.

Le raisonnement de performance ne tranche pas. Un modèle de performance des flux hybrides conclut que pour les applications intensives en calcul, la co-localisation offre actuellement un bénéfice de performance négligeable, tandis que l'intégration étroite demeure critique pour les tâches temps réel [74]. Appliqué seul, ce résultat désignerait T1 comme nominal pour tout ce qui n'est pas temps réel.

Ce que le raisonnement de performance ne voit pas. INV-1 borne le gain de la co-localisation par la part de la communication dans le temps total. Cette borne porte sur le **temps d'exécution d'une charge**. Or l'objet de cette architecture n'est pas le temps d'exécution d'une charge : c'est l'**ordonnancement d'un ensemble de charges sur une ressource instable**. Ce sont deux grandeurs différentes, et INV-1 ne dit rien de la seconde. Ce que T2 apporte n'est pas de la latence en moins, c'est de l'**information et du contrôle en plus** : une file unique au lieu de deux [32], la possibilité d'inscrire la fenêtre d'étalonnage comme réservation, un préavis qui ne dépend plus de ce que le fournisseur publie, et une télémétrie qui couvre la ressource elle-même.

Décision : T2 nominal, T1 et T3 en variantes déclarées. Le critère n'est pas la performance mais la **réalisabilité des exigences** : trois exigences sont réalisables en T2 et ne le sont pas, ou pas entièrement, en T1. Une plateforme qui ne peut pas consentir l'investissement d'hébergement opère en T1, et l'architecture reste valide — elle perd la réservation des fenêtres, dégrade le préavis et, sous les valeurs d'hypothèse du § 7.3, voit la règle R6 écarter du placement direct toute ressource à file opaque : la confiance accordée aux estimations d'attente y est le premier paramètre qu'un centre aurait à recalibrer. Cette dégradation est **déclarée** plutôt que masquée, et le § 9.1 porte les deux exigences concernées avec un verdict conditionnel à la topologie.

6.3 L'écart de niveau, et la projection qui le comble

Le constat déterminant de la revue est un **écart de niveau**. QDMI porte le signal — les propriétés changeantes existent et sont lisibles [8]. QRMI porte l'ordonnancement — et son cycle de vie documenté compte trois temps [9], [10]. *Inférence de l'auteur, que les sources n'énoncent pas comme une limite* : ce cycle décrit une occupation, non une qualité, de sorte qu'il n'offre pas de place à un état « acquise mais dégradée ». Rien n'établit qu'une extension du cycle serait impossible; ce qui est établi, c'est qu'aucune n'est publiée.

La seconde strate impose de préciser ce constat plutôt que de le répéter. Des fragments de projection existent : un instantané de dispositif lié contractuellement au moment de la réclamation [43], des scores scalaires dérivés des données d'étalonnage pour choisir un placement [44], [45], [52], une exécution conditionnée à l'évaluation dynamique des ressources [47]. Le § 2.6 les examine un à un — et la lecture intégrale des quatre, conduite en révision, confirme l'examen. Aucun ne publie ce que l'ordonnancement d'un centre exige : un état daté porté par un cycle de vie total, dont la péremption est un événement, consommé par le gestionnaire de ressources. Le canon distribué classique fournit le précédent du **mécanisme** (§ 2.4) — état publié, daté,

consommé par un appareil — sans jamais en publier ce **contenu**. La revendication est donc resserrée, non abandonnée — et c’est sous cette forme resserrée qu’elle s’expose à RÉF-2, dont elle a passé le premier test.

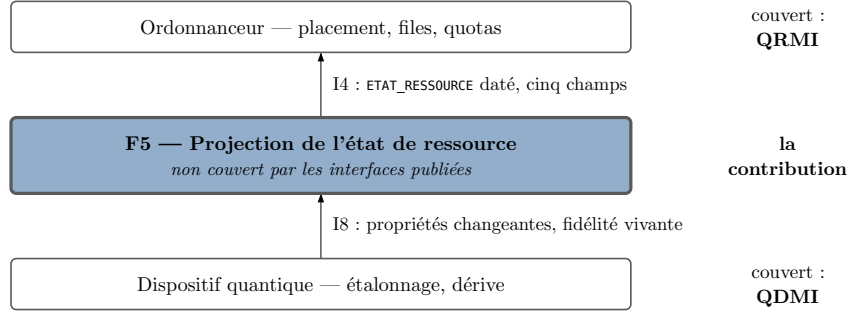


Fig. 3 — Position de la projection. Les deux interfaces publiées couvrent les extrémités; le lien entre elles n’est publié comme objet de première classe de l’ordonnancement par aucune source des deux strates (§ 2.6). C’est la contribution architecturale centrale.

Décision de conception. L’architecture réutilise une interface de type QDMI et une interface de type QRMI sans en redéfinir la sémantique, et **ajoute** la projection. L’option d’une abstraction distincte a été écartée pour une raison de coût système : elle ajouterait une interface de plus à un paysage dont un survey de cent sept publications relève déjà l’absence de normalisation [75]. Le coût de la fragmentation dépasserait le gain de la cohérence interne.

La conséquence est assumée et défavorable. C’est la plateforme, et non le fournisseur, qui porte la responsabilité de dériver l’état depuis les propriétés. Chaque famille de *backend* exige donc sa règle de projection, ce qui rend la portabilité plus difficile — et le § 9.3 concède qu’une pile existante fait mieux sur ce point précis.

6.4 Vue dynamique : machine d’états

Quatre états et un niveau de dégradation continu porté par la métrique de fidélité, plutôt que par des sous-états. Le choix a deux raisons. D’abord, le seuil qui sépare « marginalement dégradé » d’« inutilisable » dépend de l’exigence de précision de la charge, qui n’est pas connue de la ressource : porter le niveau par la métrique place le seuil **dans la politique**, où il appartient. Ensuite, la robustesse au régime : la cadence de réétalonnage peut descendre à l’échelle de la milliseconde en boucle fermée [2], et une hiérarchie de sous-états d’étalonnage deviendrait alors un coût de transition sans usage.

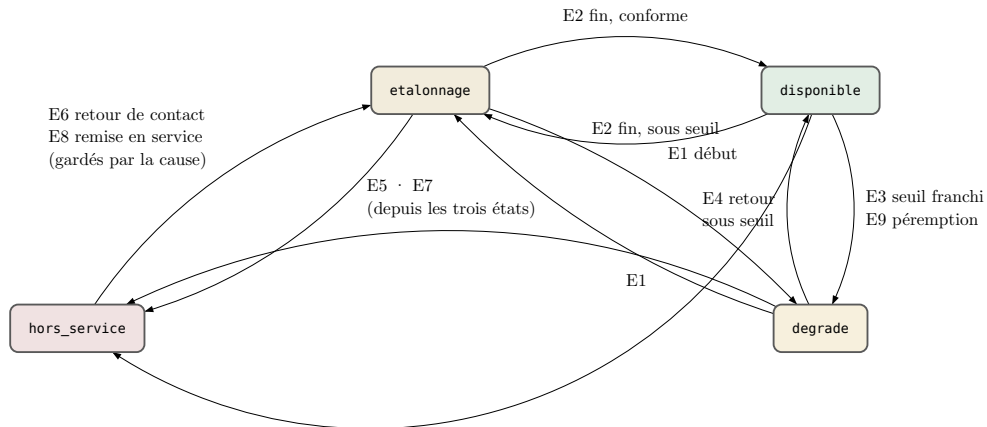


Fig. 4 — Machine d’états du QPU. Événements : E1 début d’étalonnage · E2 fin d’étalonnage · E3 franchissement du seuil de dérive · E4 retour sous le seuil après requalification · E5 perte de contact · E6 retour de contact · E7 retrait administratif · E8 remise en service · E9 préemption de la métrique de fidélité. E5 et E7 mènent à **hors_service** depuis les trois autres états; une seule des trois arêtes porte le libellé. Les sorties E6 et E8 de **hors_service** sont gardées par la cause enregistrée. Les transitions sans effet ne sont pas tracées : la table de transitions ci-dessous les porte, gardes comprises.

La table de transitions est **totale** : quatre états, neuf événements, trente-six cases, aucune vide.

État \ Évén.	E1	E2	E3	E4	E5	E6	E7	E8	E9
D — disponible	→ É	—	→ G	—	→ H	—	→ H	—	→ G
É — etalonnage	—	→ D si conforme, sinon → G	—	—	→ H	—	→ H	—	—
G — degrade	→ É	—	maj. fidélité	→ D	→ H	—	→ H	—	—
H — hors_service	—	—	—	—	—	→ É si cause = perte_contact, sinon —	maj. cause	→ É si cause = retrait, sinon —	—

Tableau 8 — Table de transitions, états abrégés par leur initiale. « — » signifie *sans effet*, non *indéfini* : chacune des trente-six cases est renseignée. Les deux sorties de H sont gardées par la cause enregistrée — *perte_contact* pour une entrée par E5, *retrait* pour une entrée par E7.

Trois points de conception sont portés par cette table.

Le retour de service passe toujours par etalonnage, jamais directement par disponible (cases E6 et E8 depuis hors_service). INV-2 borne dans le temps la validité d’une métrique de fidélité : une ressource revenue d’une absence n’en a donc aucune valide, et la déclarer disponible reviendrait à publier une information périmée comme fraîche.

La sortie de hors_service est gardée par la cause enregistrée. L’état confond deux origines — perte de contact (E5) et retrait administratif (E7) — et une sortie non gardée les confondrait aussi : la séquence E7, E5, E6 remettrait en route, sur un simple retour de contact, une ressource retirée administrativement. Les gardes ferment cette séquence : E6 ne fait sortir que si la cause enregistrée est *perte_contact*, E8 que si elle est *retrait*, et E7 survenant en hors_service met à jour la cause — le retrait prime, et une ressource retirée ne revient que par E8. Le cas résiduel est déclaré plutôt que masqué : une remise en service prononcée alors que le contact est aussi perdu place la ressource en *etalonnage*, d’où E5 la ramène aussitôt en hors_service, cette fois avec *perte_contact* pour cause.

E9 fait passer de disponible à degrade. Ce n’est pas une dégradation physique mais une dégradation de la **connaissance**. Confondre les deux dans un même état est délibéré et discutable; la justification est ce que la politique en fait — dans les deux cas, la ressource ne peut pas être choisie sur la foi d’une fidélité annoncée. C’est aussi un point de fragilité assumé : l’information qui distingue les deux origines n’existe que dans le champ *cause*, et le § 8.5 en fait la première étape de la procédure de requalification.

L’état publié porte cinq champs : la valeur d’état, son horodatage, la métrique de fidélité en vigueur avec sa date de validité, **la cause du dernier changement**, et la durée prévue de l’état courant. La cause est ce qui permet de distinguer une entrée en *etalonnage* planifiée d’une entrée en *degradation*, donc de ne pas ouvrir d’incident sur la première.

6.5 Analyse systémique : boucles et effets émergents

Une vue fonctionnelle montre ce qui est branché sur quoi; elle ne montre pas ce que la composition produit. Quatre boucles de rétroaction traversent l’architecture. La convention de polarité est celle des diagrammes causaux : un tel diagramme encode l’existence d’un lien et sa polarité, et rien de plus. **Aucune affirmation quantitative n’en est tirée.**

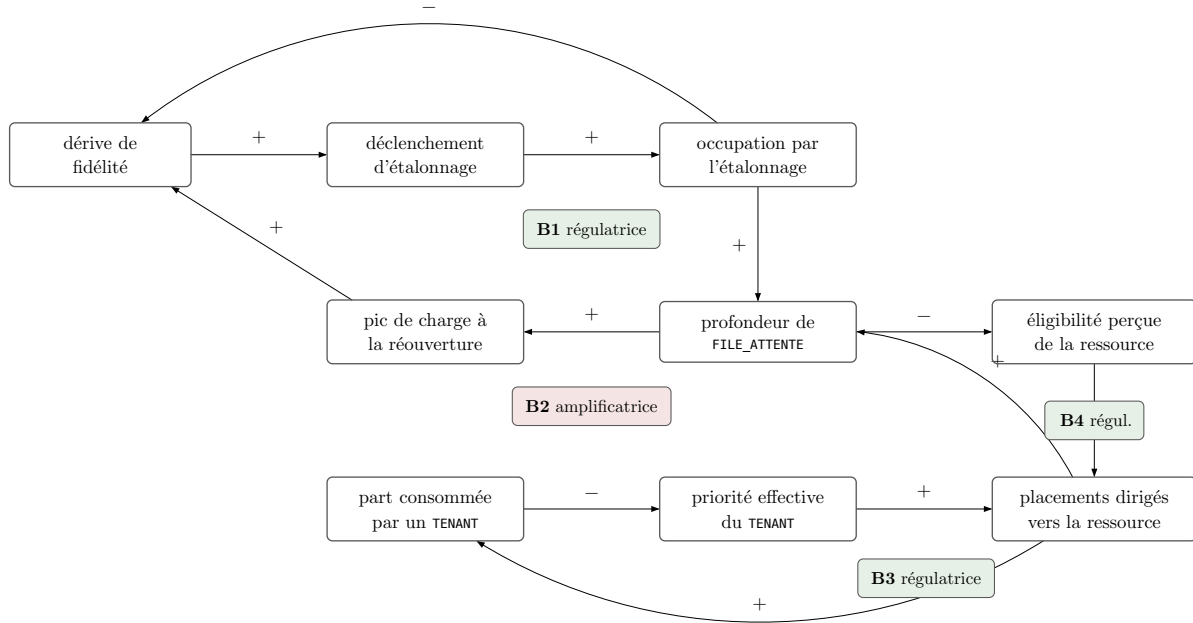


Fig. 5 — Diagramme causal des quatre boucles. B1, B3 et B4 sont régulatrices; B2 est amplificatrice. Le lien « pic de charge → dérive » qui ferme B2 est une **hypothèse déclarée**, non établie par les sources.

B1 — étalonnage (régulatrice). La dérive déclenche l'étalonnage, qui occupe la ressource, ce qui réduit la dérive. C'est la boucle qui rend la ressource utilisable dans la durée, et c'est aussi celle qui consomme sa disponibilité.

B2 — pic post-fenêtre (amplificatrice). L'occupation par l'étalonnage allonge la file, qui produit un pic à la réouverture, qui accélère la dérive, qui rapproche l'étalonnage suivant. **Le lien « pic → dérive » est une hypothèse déclarée** : les mesures disponibles établissent l'existence de la dérive et ses échelles temporelles [1], [2], non sa dépendance à l'intensité d'usage. Cette boucle est donc portée comme risque à surveiller — une ligne de télémétrie existe au § 8.3 pour la détecter — et non comme mécanisme établi. Le § 10.2 en fait une condition de réfutation.

B3 — équité (régulatrice) et **B4 — évitement de file (régulatrice)** jouent le même rôle correcteur, l'une sur la part consommée par un tenant, l'autre sur la profondeur de file observée.

Quatre effets émergents indésirables résultent de la composition, et aucun n'est un défaut d'une fonction particulière. Le quatrième arrive avec le septième archétype (§ 5.2) et il est porté en hypothèse.

Famine B3 régule la part consommée par le haut mais ne garantit rien par le bas. Un tenant de classe basse dont les charges sont systématiquement dépassées n'atteint jamais sa borne, donc B3 ne se déclenche jamais en sa faveur. **La borne supérieure ne produit pas de plancher**, et le § 8 doit porter un mécanisme distinct.

Oscillation de placement B4 réagit à la profondeur de file observée. Si plusieurs décisions sont prises sur la même observation avant que la file ne la reflète, elles se dirigent toutes vers la ressource momentanément la moins chargée, qui devient la plus chargée. C'est le comportement classique d'une boucle régulatrice à retard; la mitigation appartient à la politique, non à l'architecture.

Synchronisation des fenêtres Si plusieurs QPU d'une flotte étalonnent selon une périodicité voisine, leurs fenêtres peuvent se synchroniser sous l'effet de B2. La plateforme se retrouverait alors sans aucune ressource quantique par intermittence — situation que l'exigence de dégradation gracieuse rend survivable, mais que personne ne souhaite.

Résonance des consommateurs autonomes L'oscillation de placement est une boucle régulatrice à retard : plusieurs décisions prises sur la même observation avant que la file ne la reflète. Des agents

autonomes (§ 5.2) qui consomment la même télémétrie publiée raccourcissent ce retard et synchronisent leurs réactions — ce qui était un effet de bord occasionnel sous soumission humaine devient un régime plausible sous soumission machine. Les mitigations existantes s’appliquent telles quelles : la décision par campagne (§ 7.5) réduit le nombre de décisions exposées, et le point de contrôle ne se déclenche que sur événement d’état, jamais sur observation de file (§ 7.4). L’ampleur résiduelle est une *hypothèse déclarée*, qu’aucune source ne mesure; `taux_report` et `profondeur_file` (§ 8.3) sont les lignes qui la rendraient observable.

7 La fonction de délégation

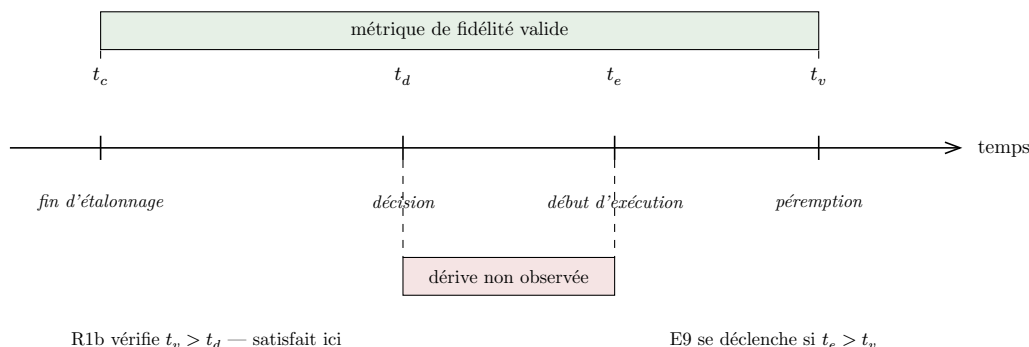
7.1 Cinq critères de décision

Chacun porte un nom, une unité, une direction d’optimisation et une source de valeur. La contrainte structurante est INV-4 : **aucun critère ne peut dépendre d’un signal disponible seulement après l’exécution.**

Critère	Unité	Direction	Source de la valeur
C1 temps de complétion estimé	s	minimiser	estimation d’attente (I11) et estimation de durée issue de la caractérisation — <i>estimateurs</i>
C2 fidélité attendue	[0, 1]	maximiser	métrique de fidélité en vigueur (I4), pénalisée par la profondeur du circuit
C3 coût	unité de compte	minimiser	tarif de la ressource $\times$ occupation estimée
C4 disponibilité attendue	[0, 1]	maximiser	état publié (I4) et fenêtres d’étalonnage réservées sur l’horizon (I10)
C5 consommation classique	cœur $\cdot$ h	minimiser	ressources classiques retenues pendant l’attente et l’exécution quantiques

Tableau 9 — Les cinq critères de décision. Tous sont disponibles avant l’exécution — c’est ce qui se vérifie par lecture d’une colonne.

Trois grandeurs qu’on aimerait connaître sont **structurellement absentes**, et leur absence est traitée plutôt qu’ignorée. La fidélité qu’une exécution donnée obtiendra sur un dispositif donné à un instant donné : les mesures établissent l’existence de la dérive [1], [2], pas une loi de prédiction. La profondeur d’une file opaque : aucun modèle ne prédit ce qui n’est pas observable (INV-5). La dérive à venir entre la décision et l’exécution : c’est l’écart temporel que rend visible la Fig. 6 sans le combler.



R1b vérifie $t_v > t_d$ — satisfait ici

E9 se déclenche si $t_e > t_v$

Fig. 6 — Chronologie d’une décision. La règle R1b vérifie que la métrique est encore valide à l’instant de la décision, mais rien n’observe la dérive entre la décision et le début de l’exécution. C’est l’écart que la politique déclare sans le combler, et que l’événement E9 rend visible s’il est franchi.

Méthode d’agrégation. Les unités étant incommensurables, l’agrégation procède en trois temps : filtrage d’éligibilité par contraintes dures, normalisation min-max **sur l’ensemble éligible de la décision courante**, puis somme pondérée. Le filtrage précède la comparaison parce qu’une ressource inéligible ne se compense pas par un bon score ailleurs — une fidélité insuffisante ne se rachète pas par un temps court.

Ce que cette forme a de connu — et de connu comme fragile. La somme pondérée sur des valeurs normalisées par l'ensemble des candidats est une méthode documentée de la décision multicritère, pathologies comprises : la plus étudiée est le **renversement de rang** — l'entrée ou la sortie d'un candidat change les valeurs normalisées de tous les autres et peut inverser l'ordre des restants [76]. La politique l'assume pour une raison déclarée : la normalisation sur l'ensemble éligible ne requiert aucune échelle absolue, là où des échelles de référence fixes par critère exigeraient précisément la calibration que la méthode ne peut pas produire (§ 3.3). Deux dispositions en bornent l'effet : la décision par campagne (§ 7.5) réduit le nombre de décisions exposées au phénomène, et la publication des alternatives avec leur score rend tout renversement observable après coup. Le passage à des échelles de référence fixes — qui atténuerait du même coup l'oscillation de placement du § 6.5 — est la variante déclarée qu'une calibration rendrait préférable; elle est le deuxième réglage attendu d'une plateforme réelle, après les bornes du § 7.6.

Les arbitrages sont dans les poids, et les poids sont publiés. Un arbitrage non publié est un arbitrage non contestable.

Classe de service	w_1 temps	w_2 fidélité	w_3 coût	w_4 disponibilité	w_5 consommation classique
interactive	0,40	0,20	0,05	0,30	0,05
standard	0,25	0,30	0,15	0,20	0,10
precision	0,10	0,55	0,10	0,15	0,10

Tableau 10 — Pondérations par classe de service. *Hypothèse de travail pour les quinze valeurs* : aucune source de la base ne publie de pondération employée en exploitation (verrou V6).

Ce que ces trois lignes disent est lisible sans calcul : une charge interactive privilégie le délai et la certitude de l'obtenir; une charge de précision privilégie la fidélité au point d'accepter d'attendre; la classe standard équilibre.

7.2 Spécification du mécanisme

La politique est déterministe — mêmes entrées, même sortie — et **totale** : la seule source potentielle d'indétermination, l'égalité de scores, est levée par un départage qui se termine sur l'identifiant de ressource, ordre strict et stable. Seize entrées typées l'alimentent, six sorties en résultent.

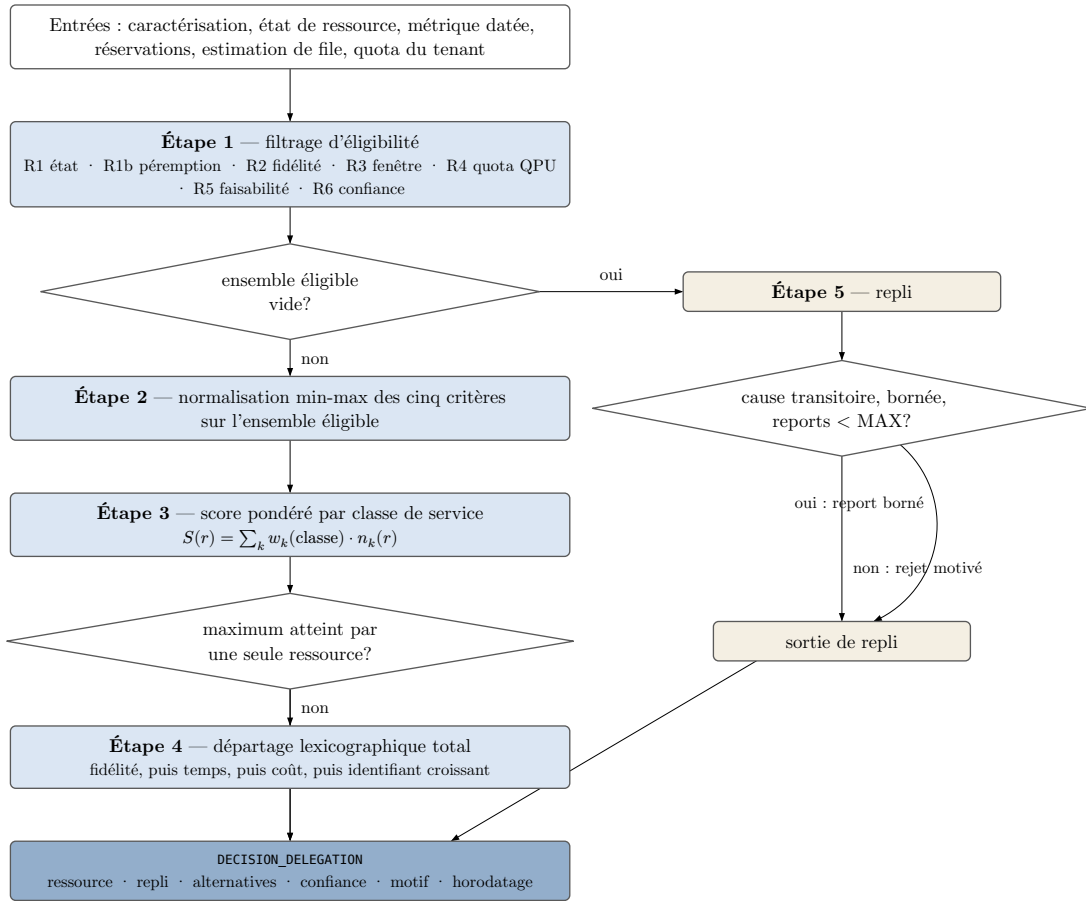


Fig. 7 — Flux de la politique de délégation. Le repli n'est pas un cas d'erreur : c'est une branche spécifiée, dont les deux issues sont énumérées et bornées.

POLITIQUE_DELEGATION(charge, classe, ressources, tenant, maintenant) :

```

# Étape 1 – filtrage d'éligibilité (contraintes dures)
pour chaque r dans ressources :
    si r.etat ∉ {disponible, degrade} : rejeter r, motif R1
    si r.fidelite_valide_jusqu_a < maintenant : rejeter r, motif R1b
    si fidelite_attendue(r, charge) < charge.precision_requise : rejeter r, motif R2
    si chevauche(r.fenetres, [maintenant,
        maintenant + r.attente_estimee + r.duree_estimee]) : rejeter r, motif R3
    si r.type == QPU et tenant.part_consommee ≥ 1,0
        et classe ≠ precision : rejeter r, motif R4
    si r.largeur_max < charge.largeur : rejeter r, motif R5
    si min(conf(r.attente), conf(r.duree)) < SEUIL_CONFIANCE : rejeter r, motif R6
    sinon : ajouter r à éligibles

# Repli – l'ensemble éligible est vide
si éligibles est vide :
    retourner REPLI(charge, ressources, motifs_de_rejet, maintenant)

# Étape 2 – normalisation min-max sur l'ensemble éligible
pour chaque critère k, avec bas = min v_k et haut = max v_k sur éligibles :
    si haut == bas : n_k(r) = 1,000
    sinon si direction(k) == max : n_k(r) = (v_k(r) - bas) / (haut - bas)
    sinon : n_k(r) = (haut - v_k(r)) / (haut - bas)
  
```

```

# Étape 3 – score pondéré
S(r) =  $\sum_k w_k(\text{classe}) \cdot n_k(r)$ 

# Étape 4 – sélection et départage total
gagnants = { r dans éligibles : S(r) == max S }
si |gagnants| == 1 : retenue = l'unique élément
sinon : départage – 1. fidélité attendue la plus haute
                2. à égalité, temps de complétion estimé le plus bas
                3. à égalité, coût le plus bas
                4. à égalité, plus petit identifiant en ordre lexicographique

confiance = min(conf(retenue.attente), conf(retenue.duree))
retourner DECISION(retenue, repli = aucun, alternatives = {(r, S(r))},
                  confiance, motif, maintenant)

```

Trois points sur lesquels la lecture peut buter sont des choix, non des omissions.

La règle R4 ne porte que sur les ressources quantiques, et exempte la classe *precision* de la borne de quota. La borne mesure la consommation de la ressource rare : un tenant qui l'a atteinte reste plaçable sur ressource classique, et étendre le rejet aux ressources classiques bloquerait des charges qu'un simulateur peut servir. Quant à l'exemption : sans elle, une charge de classe *precision* serait rejetée dès que son tenant atteint sa borne, alors qu'elle n'a typiquement aucune autre ressource capable de la servir; avec elle, la borne devient un plafond souple pour cette seule classe. C'est un arbitrage entre équité et utilité du service, ici tranché en faveur du second — décision contestable, ce qui est la raison de la publier. Le § 9.2 montre que c'est aussi ce qui fait échouer partiellement le scénario de multi-tenance.

La règle R6 s'applique ressource par ressource, non à la décision entière. Écarter globalement une décision parce qu'un candidat est mal estimé pénaliserait les candidats bien observés pour l'opacité de leurs concurrents : la présence d'une seule file opaque dans l'ensemble éligible suffirait à interdire le placement direct sur toutes les ressources, y compris les mieux observées. La confiance est donc une contrainte d'éligibilité comme les autres — elle écarte la ressource qu'on ne sait pas estimer, et elle seule.

La comparaison `haut == bas` renvoie 1,000 et non 0,000. Quand tous les candidats ont la même valeur sur un critère, ce critère ne discrimine pas, et lui attribuer 0 pénaliserait arbitrairement l'ensemble.

Les six sorties — ressource retenue, forme du repli, alternatives écartées avec leur score, confiance, motif ayant tranché, horodatage — réalisent l'exigence d'observabilité de la décision. Le motif rend l'arbitrage lisible sans le recalculer.

7.3 Traitement de l'incertitude

Deux entrées sont des estimateurs, l'attente et la durée. Chacune porte une confiance dans $[0, 1]$, et **la confiance de la décision est la plus faible des confiances mobilisées pour la ressource retenue** — une chaîne ne vaut pas plus que son maillon le plus faible, et retenir la moyenne masquerait un estimateur défaillant derrière un bon.

La confiance de l'estimateur d'attente est **fonction déclarée de l'opacité** de la file : 0,90 sur une file observable, 0,40 sur une file opaque (*hypothèses de travail*). Combinée à la règle R6, cette valeur a une conséquence forte et voulue : 0,40 étant sous le seuil, **une ressource dont la file est entièrement opaque est écartée au filtrage** — l'application directe d'INV-5, et elle s'applique ressource par ressource : l'opacité d'une candidate n'empêche pas d'en retenir une autre, mieux observée. La confiance de l'estimateur de durée suit la profondeur d'historique comparable, avec les mêmes valeurs.

Le seuil `SEUIL_CONFIAANCE = 0,50` (*hypothèse de travail*) est choisi pour une raison lisible : en deçà, l'estimateur informe moins qu'un tirage entre deux options, et un placement fondé sur lui serait un placement fondé sur rien.

Une conséquence de ces valeurs mérite d’être nommée plutôt que découverte : sous les hypothèses de travail, R6 **dégénère en règle binaire** — l’exclusion des files opaques —, et la médiation par un score continu ne discrimine rien. La forme continue n’est pas décorative pour autant : la confiance de durée suit une profondeur d’historique qui varie continûment, et le seuil est un paramètre de calibration qu’un centre peut abaisser pour réadmettre les files opaques au filtrage — c’est le premier des réglages que le § 6.2 annonçait pour la topologie T1.

7.4 Repli

Le repli n’est atteint que sur ensemble éligible vide, et il est conservateur : il ne tente jamais un placement à faible confiance en espérant qu’il passe — la règle R6 a déjà écarté ces placements au filtrage. Il n’a pas non plus de branche « repli classique » : une ressource classique — simulateur compris — est candidate de première classe au filtrage, et quand toutes les quantiques sont écartées, c’est le chemin nominal qui la retient. Une branche distincte la compterait deux fois. Le repli proprement dit a deux issues.

1. **Report borné** — s’il existe une ressource dont le seul motif de rejet est transitoire et borné (état, péremption de métrique, fenêtre planifiée) et si le compteur de reports de la charge est sous `MAX_REPORTS = 3` (*hypothèse de travail*). La borne est nécessaire : sans elle, une charge peut être reportée indéfiniment, ce qui est la famine sous un autre nom.
2. **Rejet motivé** — sinon, avec la liste des motifs de rejet par ressource.

L’exécution spéculative sur deux ressources a été écartée sur invariant. Elle double l’occupation d’une ressource rare pour couvrir une incertitude, et INV-6 impose le coût d’opportunité comme critère quand une charge placée en exclut une autre. L’option resterait défendable sur une ressource abondante; elle ne l’est pas ici.

Ce que ce choix coûte. Un taux de report plus élevé qu’une politique optimiste, donc un délai moyen plus long pour les charges dont la ressource préférée est instable. La table de télémétrie du § 8.3 exige la ligne qui permettra de le mesurer — c’est la seule manière honnête de porter un compromis qu’on ne peut pas chiffrer d’avance.

Le point de contrôle de campagne. La décision est prise une fois, avant l’exécution — INV-4 l’impose —, et la première version laissait ce choix sans recours : rien ne réévaluait une charge en cours d’exécution, ce que le rejeu du § 9.2 payait au scénario de dégradation imprévue. La révision spécifie le mécanisme qui manquait, sans toucher à l’invariant. Pour une charge **itérative** — campagne variationnelle en tête —, un événement E3, E5 ou E9 sur la ressource en cours d’exécution rouvre la décision pour le reliquat de la campagne : les itérations restantes repassent par la politique comme un report, compté dans `MAX_REPORTS`. Le point de contrôle ne consomme que des signaux pré-exécution des itérations restantes — INV-4 est respecté — et ne se déclenche que sur événement d’état, jamais sur observation de file — l’oscillation du § 6.5 n’est pas réintroduite par la porte de service. Un noyau monolithique reste hors de portée : rien ne peut réévaluer une exécution qui ne rend la main qu’à sa fin, et le § 9.2 porte ce résidu comme tel.

7.5 Déroulé vérifiable

La politique se vérifie sans matériel. Voici deux déroulés complets — un lecteur qui obtient un autre résultat a réfuté la spécification (condition RÉF-6, § 10.2). Depuis la révision, un script de rejeu versionné (`rejeu-politique.py`, voir Disponibilité) implémente la politique, reproduit les deux déroulés et l’analyse de sensibilité, et vérifie la totalité de la table de transitions du § 6.4, gardes comprises : la vérification à la main reste possible, elle n’est plus le seul chemin.

Parc, commun aux deux déroulés (*hypothèses de travail*) : QPU-A (40 qubits, disponible, fidélité 0,97, tarif élevé), QPU-B (32 qubits, degrade, 0,88, moyen), SIM-GPU (34 qubits, disponible, 1,00 par construction, bas). Les charges sont peu profondes, la pénalité de profondeur vaut 1, donc la fidélité attendue coïncide avec la métrique.

7.5.1 Déroulé A — noyau fixe, classe standard

Charge : type noyau fixe, largeur 30 qubits, précision requise 0,85, classe `standard`.

Étape 1 — les trois candidates sont éligibles. QPU-B l'est bien qu'en état `degrade` : la règle R1 admet cet état, et sa fidélité 0,88 dépasse l'exigence 0,85. Les trois files sont observables et l'historique comparable suffisant : les confiances valent 0,90 et R6 ne rejette rien. C'est le point où une machine d'états binaire aurait déjà perdu de l'information.

	C1 (s)	C2	C3	C4	C5
QPU-A	900	0,97	120	0,95	0,5
QPU-B	300	0,88	60	0,80	0,5
SIM-GPU	5400	1,00	40	0,99	12,0

Tableau 11 — A — valeurs des critères (*hypothèses*).

	n_1	n_2	n_3	n_4	n_5
QPU-A	0,882	0,750	0,000	0,789	1,000
QPU-B	1,000	0,000	0,750	0,000	1,000
SIM-GPU	0,000	1,000	1,000	1,000	0,000

Tableau 12 — A — valeurs normalisées.

Étape 2 — étendues : $C1 \rightarrow 5100$, $C2 \rightarrow 0,12$, $C3 \rightarrow 80$, $C4 \rightarrow 0,19$, $C5 \rightarrow 11,5$.

Étape 3 — score avec les poids standard (0,25 ; 0,30 ; 0,15 ; 0,20 ; 0,10) :

$$S(\text{QPU-A}) = 0,25 \cdot 0,882 + 0,30 \cdot 0,750 + 0,15 \cdot 0,000 + 0,20 \cdot 0,789 + 0,10 \cdot 1,000 = \mathbf{0,703}$$

$$S(\text{QPU-B}) = 0,25 \cdot 1,000 + 0,30 \cdot 0,000 + 0,15 \cdot 0,750 + 0,20 \cdot 0,000 + 0,10 \cdot 1,000 = \mathbf{0,463}$$

$$S(\text{SIM-GPU}) = 0,25 \cdot 0,000 + 0,30 \cdot 1,000 + 0,15 \cdot 1,000 + 0,20 \cdot 1,000 + 0,10 \cdot 0,000 = \mathbf{0,650}$$

Décision : QPU-A, maximum unique, pas de départage; alternatives conservées avec leur score; confiance 0,90 (files observables, historique comparable suffisant).

Ce qui rend ce déroulé intéressant n'est pas le gagnant, c'est l'écart : 0,053 entre QPU-A et le simulateur. Un poids w_2 porté de 0,30 à 0,40 aux dépens de w_1 (0,25 $\rightarrow$ 0,15) fait basculer la décision vers le simulateur, 0,750 contre 0,690. **L'arbitrage réel est dans les poids** — c'est précisément pourquoi ils sont publiés plutôt qu'enfouis dans une implémentation.

7.5.2 Déroulé B — campagne variationnelle, classe interactive

Charge : type variationnel, largeur 30, précision requise 0,85, classe `interactive`, budget de 40 itérations. **La décision porte sur la campagne entière, non sur chaque itération** : c'est ce que prescrit la politique, et c'est ce qui empêche l'oscillation de placement du § 6.5.

Les critères sont agrégés sur les 40 itérations (*hypothèses de travail*) : QPU-A (3600 s ; 0,97 ; 480 UC ; 0,95 ; 8,0 cœur · h), QPU-B (1800 ; 0,88 ; 240 ; 0,80 ; 4,0), SIM-GPU (14 400 ; 1,00 ; 160 ; 0,99 ; 40,0). Les étendues deviennent $C1 \rightarrow 12\,600$, $C2 \rightarrow 0,12$, $C3 \rightarrow 320$, $C4 \rightarrow 0,19$, $C5 \rightarrow 36,0$, et les valeurs normalisées de QPU-A sont (0,857 ; 0,750 ; 0,000 ; 0,789 ; 0,889).

Avec les poids `interactive` (0,40 ; 0,20 ; 0,05 ; 0,30 ; 0,05) :

$$S(\text{QPU-A}) = 0,40 \cdot 0,857 + 0,20 \cdot 0,750 + 0,05 \cdot 0,000 + 0,30 \cdot 0,789 + 0,05 \cdot 0,889 = \mathbf{0,774}$$

$$S(\text{QPU-B}) = 0,40 \cdot 1,000 + 0,20 \cdot 0,000 + 0,05 \cdot 0,750 + 0,30 \cdot 0,000 + 0,05 \cdot 1,000 = \mathbf{0,488}$$

$$S(\text{SIM-GPU}) = 0,40 \cdot 0,000 + 0,20 \cdot 1,000 + 0,05 \cdot 1,000 + 0,30 \cdot 1,000 + 0,05 \cdot 0,000 = \mathbf{0,550}$$

Décision : QPU-A. Le changement de classe a élargi l'écart avec le simulateur — de 0,053 en classe `standard` à 0,224 en classe `interactive` — parce que la classe `interactive` pénalise lourdement le temps de complétion, où le simulateur est le plus faible. Deux charges de même forme reçoivent donc des décisions de robustesse très différente selon leur classe, et c'est un comportement voulu, non un effet de bord.

7.6 Sélection de la stratégie de partage

Le verrou V5 est qu'aucune règle publiée ne choisit entre les trois stratégies mesurées par [13] pour une charge donnée. La règle suivante comble ce vide en s'appuyant sur le seul discriminant que les mesures fournissent : le déséquilibre entre part quantique et part classique. Soit ρ le rapport de la durée quantique estimée à la durée classique estimée, calculable à la caractérisation.

Condition	Stratégie retenue	Fondement
charge parallèle	décomposition de flux	les charges sont indépendantes, la décomposition s'applique sans coût de coordination, et c'est le régime où elle réduit le plus la consommation classique [13]
$0,5 \leq \rho \leq 2,0$	malléabilité	régime équilibré, celui où la source situe le domaine de validité de la malléabilité [13]
$\rho < 0,5$ ou $\rho > 2,0$	multiplexage temporel	régime de déséquilibre marqué, celui que la source associe au multiplexage [13]

Tableau 13 — Règle de sélection entre stratégies de partage.

Les bornes 0,5 et 2,0 sont des hypothèses de travail. La source qualifie les régimes (« équilibré », « déséquilibre marqué ») sans publier de seuil numérique; le choix d'un facteur 2 est un choix de symétrie, non un résultat. C'est le premier réglage qu'une plateforme réelle aurait à faire, et l'un des rares où la mesure requise est à sa portée immédiate.

Portée de la règle. Deux précisions la bordent. D'abord, les trois conditions du tableau s'évaluent dans l'ordre : la première ligne dont la condition est vraie l'emporte — une charge parallèle en régime équilibré relève donc de la décomposition de flux. Ensuite, la règle n'arbitre qu'entre les trois stratégies **temporelles** mesurées par [13]. La multi-programmation spatiale — co-placer plusieurs circuits sur un même dispositif, mesurée depuis 2019 [14], [15], [53], [54], [55] — est une quatrième famille, déclarée hors de portée : son domaine de validité se définit par la largeur des circuits, la topologie et la diaphonie, non par le rapport ρ , et son intégration exigerait de spécifier l'isolation entre co-locataires, avec les risques adverses documentés en multi-tenance [67]. C'est une exclusion déclarée, non une omission — et le premier élargissement que la règle devrait subir si la rareté du QPU persiste (INV-6).

7.7 Domaine de validité

La politique est spécifiée pour le régime actuel, où le QPU est une ressource rare et bruitée dont la métrique de fidélité se périme. Deux conditions de bascule la rendraient caduque. Si l'étalonnage cessait d'occuper la ressource, les règles R1 et R3 perdraient leur objet. Si le régime tolérant aux fautes devenait la norme, la fidélité cesserait d'être un critère de placement pour devenir une propriété garantie — et les modèles de latence de décodage [33] situent cette bascule au-delà d'une amélioration d'au moins un ordre de grandeur des décodeurs actuels.

8 Exploitabilité

8.1 Sept exigences qui ne se formuleraient pas sans QPU

Q3 demande ce qui distingue l'exploitabilité d'une plateforme HPC+QPU de celle d'une plateforme classique. Le critère de sélection est réfutable : **si l'exigence se retrouve à énoncé équivalent dans la documentation d'exploitation d'une plateforme HPC sans QPU, la distinction n'existe pas.**

Exigence	Pourquoi elle ne se formule pas sur une plateforme classique
L'étalonnage est un état nominal, distinct de la mise hors service	Une ressource classique n'a pas d'état intermédiaire récurrent entre disponible et en panne. La maintenance planifiée d'un nœud existe, mais sa périodicité se mesure en mois et elle ne conditionne pas la qualité du résultat entre deux occurrences.
Chaque métrique de fidélité porte sa date de validité	Un cœur de calcul délivre le même résultat aujourd'hui et demain : il n'y a pas de métrique de qualité de sortie qui se périme.
Chaque fenêtre d'étalonnage planifiée est publiée avec un préavis	Corollaire du précédent : rien d'équivalent à annoncer sur une ressource dont la qualité ne dérive pas.
Le caractère observable de chaque file est déclaré	Sur une plateforme HPC consolidée, l'ordonnanceur possède ses files. La grille fédérée a connu l'opacité (§ 2.4) et normalisé la description des ressources [40] — sans jamais faire de l'observabilité de chaque file une propriété déclarée et exigible. L'exigence se formule ici

	parce que la non-coïncidence entre frontière de propriété et frontière du système (INV-5) redevient le cas nominal, et qu'elle porte cette fois une qualité qui dérive.
Les soumissions restent acceptées quand toutes les ressources quantiques sont indisponibles	Sur une plateforme classique, l'indisponibilité simultanée d'une classe entière est un incident majeur; ici, c'est un état traversé à chaque étalonnage.
Le résultat porte son incertitude	Le résultat d'une exécution classique est déterministe. L'incertitude d'une exécution quantique est une propriété de la ressource, pas du programme.
L'objectif de disponibilité est différencié par classe de service	Une plateforme classique différencie ses classes par la priorité et le quota, non par le niveau de disponibilité de la ressource sous-jacente, qui est homogène.

Tableau 14 — Sept exigences d'exploitabilité propres à une plateforme intégrant un QPU.

Le fondement commun des sept tient en une phrase : elles découlent toutes de ce qu'une ressource de calcul peut avoir **une qualité de sortie variable dans le temps et une part d'état non observable** — deux propriétés que le modèle d'exploitation du HPC classique ne prévoit pas.

8.2 L'étalonnage comme processus opérationnel

Le retour d'expérience d'intégration d'un QPU de 20 qubits en centre HPC décrit les ordinateurs quantiques comme des systèmes dynamiques exigeant un réétalonnage régulier, automatique et **pilotable par l'ordonnanceur HPC** [68]. C'est la seule source de la base qui énonce le principe de coordination; elle n'en donne ni la procédure ni la périodicité.

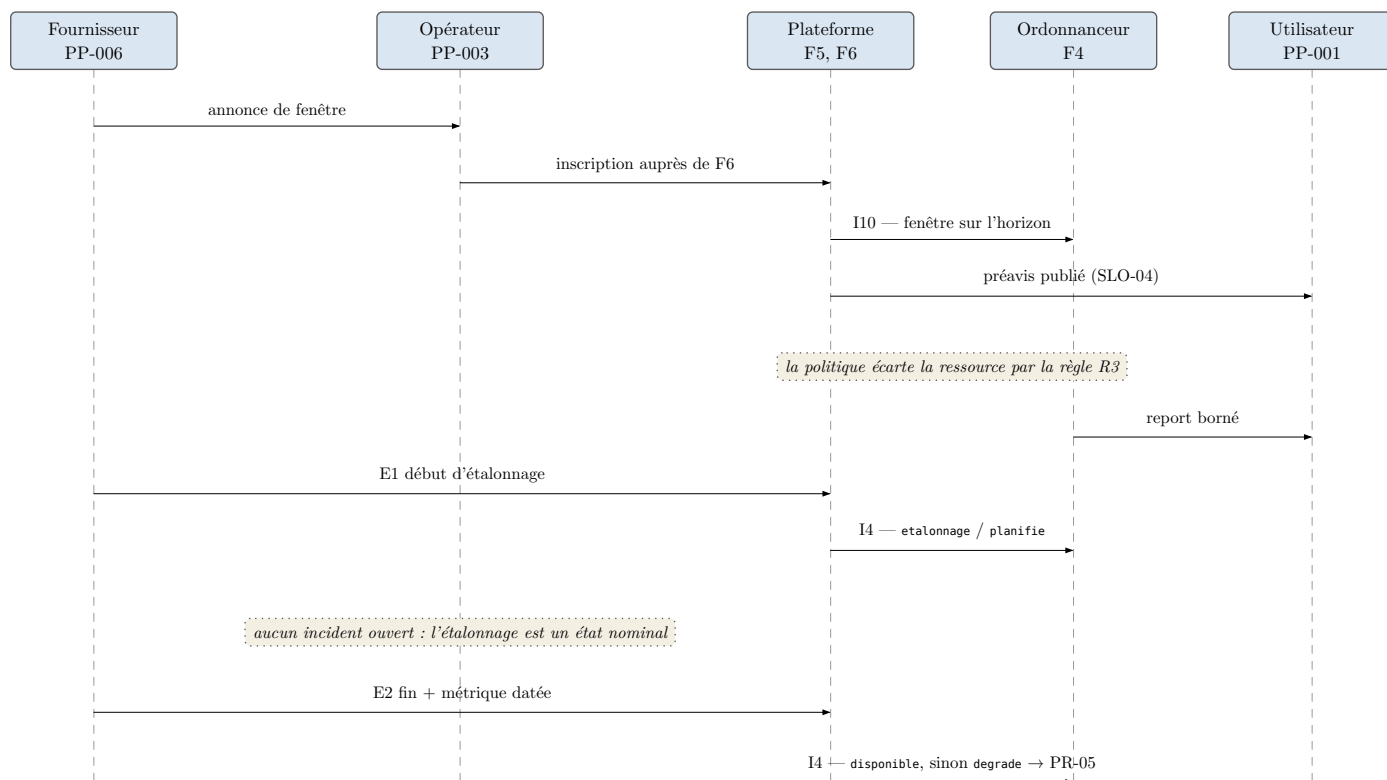


Fig. 8 — Coordination d'une fenêtre d'étalonnage planifiée (procédure PR-01). Le point clé est l'étape où la politique écarte elle-même la ressource par la règle R3 : l'opérateur ne vide pas la file à la main, ce qui garde la propagation observable.

Procédure PR-01 — planification et exécution d'une fenêtre planifiée. Neuf étapes : enregistrer la fenêtre, vérifier le préavis contre l'objectif de service et consigner l'écart le cas échéant, publier le préavis, **laisser la politique écarter la ressource par la règle R3 plutôt que vider la file à la main**, constater la transition E1 avec cause = planifie, vérifier qu'aucun incident n'a été ouvert, lire à la sortie la

métrique produite avec sa date de validité, constater le retour à `disponible` ou basculer vers la procédure de requalification, constater la reprise des charges reportées.

Critère d’arrêt : la ressource a quitté l’état `etalonnage` et une métrique postérieure à la fenêtre est publiée et aucune charge n’est restée bloquée au-delà de `MAX_REPORTS`.

Point de reprise : chaque étape est idempotente et l’état de la procédure est reconstituable depuis l’état publié et la réservation active. Aucun état n’est tenu dans la tête de l’opérateur.

Ce que PR-01 propage. Une fenêtre d’étalonnage n’est pas un événement local. Elle allonge la file (boucle B2), donc déplace des charges vers d’autres ressources (boucle B4), donc consomme le quota d’autres tenants (boucle B3). L’étape « laisser la politique écarter » existe précisément pour que ces propagations passent par les mécanismes prévus et restent observables.

La périodicité est un paramètre d’exploitation, pas une constante d’architecture. Aucune source ne la publie, et les mesures disponibles couvrent trois régimes incompatibles : dérive inter- et intra-journalière [1], dérive sub-seconde avec réétalonnage en boucle fermée à l’échelle de la milliseconde [2], réétalonnage « régulier » sans période énoncée [68].

Le cas de la fenêtre courte. Si la cadence descend à l’échelle de la milliseconde [2], PR-01 cesse d’être exécutable par un opérateur : on ne publie pas un préavis pour une fenêtre de quelques millisecondes. Deux régimes coexistent alors — les fenêtres longues et rares relèvent de la procédure; les fenêtres courtes et fréquentes deviennent, du point de vue de la plateforme, une composante du temps d’exécution plutôt qu’un événement d’ordonnancement. *Hypothèse déclarée* : le seuil qui sépare les deux est de l’ordre du temps d’exécution d’une charge typique.

8.3 Observabilité et télémétrie

Le principe de découplage entre collecte de télémétrie et exécution des charges [34] est repris tel quel : c’est ce qui permet à la fonction de collecte d’opérer **pendant** une fenêtre d’étalonnage, c’est-à-dire précisément au moment où l’on souhaite observer.

Métrique	Unité	Fréquence	Seuil d’alerte
<code>etat_ressource</code>	énumération	à chaque transition	transition vers <code>hors_service</code> non planifiée
<code>cause_transition</code>	énumération	à chaque transition	<code>cause</code> $\neq$ planifie sur entrée en <code>etalonnage</code>
<code>fidelite_courante</code>	[0, 1]	à chaque étalonnage	paramétré : sous le seuil déclaré
<code>age_metrrique_fidelite</code>	s	1 min	paramétré : au-delà de la durée de validité déclarée
<code>pente_derive</code>	fidélité/h	1 h, glissant	paramétré : facteur k de la pente médiane de la ressource
<code>coherence_etalonnage</code>	écart normalisé	à chaque étalonnage	paramétré : écart inter-cycles au-delà du seuil déclaré (§ 8.6)
<code>profondeur_file</code>	charges	1 min	paramétré : au-delà d’un plafond par ressource
<code>opacite_file</code>	booléen	à chaque changement	passage d’observable à opaque
<code>confiance_estimateur_attente</code>	[0, 1]	à chaque décision	sous <code>SEUIL_CONFIANCE</code>
<code>taux_report</code>	reports/décisions	1 h	paramétré : au-delà du plafond déclaré
<code>taux_delegation</code>	décisions QPU/total	24 h, glissant	aucun — donnée de pilotage
<code>reports_par_charge</code>	entier	à chaque report	atteinte de <code>MAX_REPORTS</code>
<code>decision_ressource_retenue</code>	identifiant	à chaque décision	aucun — traçabilité

decision_alternatives	liste	à chaque décision	aucun — traçabilité
decision_confiance	[0, 1]	à chaque décision	aucun — traçabilité
part_consommee_tenant	[0, 1]	5 min	atteinte de 1,0
part_serve_tenant	[0, 1]	24 h	paramétré : sous le plancher d'équité
preavis_fenetre	s	à chaque annonce	sous la valeur déclarée à SLO-04
duree_fenetre_effective	s	à chaque fenêtre	dépassement de la durée annoncée
disponibilite_classe	[0, 1]	24 h, glissant	paramétré : sous la cible de la classe

Tableau 15 — Table de télémétrie : vingt métriques, aucune case vide. Les seuils marqués « paramétré » sont déclarés comme tels plutôt qu'inventés; les fréquences chiffrées sont des *hypothèses de travail* à ajuster par le centre.

Trois lignes méritent d'être signalées. Les trois lignes `decision_*` réalisent à elles seules l'exigence d'observabilité de la décision — ressource retenue, alternatives écartées, niveau de confiance. Et `penetration_derive` **n'existe pour aucune exigence** : elle est là pour rendre mesurable la boucle B2, dont le lien « pic de charge → dérive » est une hypothèse déclarée. La corrélérer avec `profondeur_file` est la manière la moins coûteuse de la confirmer ou de la réfuter en exploitation. C'est la seule métrique du tableau dont la justification soit un doute plutôt qu'un besoin. Enfin `coherence_etat` est la seule ligne issue de la seconde strate : elle répond à une menace documentée sur l'intégrité de l'entrée d'étalonnage, que le § 8.6 expose.

8.4 Objectifs de service

Une ressource peu disponible ne se couvre pas d'un objectif calqué sur le HPC classique. Le seul ordre de grandeur publiquement mesuré est de 92 % sur six mois pour des utilisateurs externes d'un ordinateur quantique photonique en nuage [42] — un **plafond de crédibilité**, non une cible transposable.

Trois précédents de la seconde strate méritent d'être nommés pour être écartés : la vérification périodique de la fiabilité d'instances NISQ pour guider leur sélection [28], l'observabilité multinuage à vues différenciées par groupe d'utilisateurs [29], et des métriques utilisateur et système mesurées sur une plateforme hybride sans serveur, traces publiées à l'appui [30]. Aucun des trois ne publie d'objectif de service pour une plateforme HPC-QPU intégrée — pas davantage, dans la première strate, que l'environnement centré utilisateur qui décrit sa pile d'observabilité sans publier d'objectif chiffré [77]. Le constat du § 2.5 tient, et il tient désormais contre une base élargie.

SLO	Grandeur	Cible	Forme de la justification
SLO-01	fraction du temps où au moins un QPU éligible à la classe est disponible	paramétrée : $\geq$ une cible par classe	source [42] comme plafond; aucune valeur transposable
SLO-02	délai entre l'acceptation d'une soumission et la première décision	≤ 60 s, 95 ^e centile	dérivation : le coût de calcul est un filtrage et une somme pondérée sur une dizaine de ressources, donc indépendant de l'état du QPU
SLO-03	fraction des exécutions démarrées avec une métrique valide	100 %	dérivation : c'est la règle R1b combinée à la revérification au démarrage; toute violation révèle un défaut de la chaîne
SLO-04	délai entre la publication du préavis et le début de la fenêtre	paramétrée : $\geq$ une valeur déclarée	l'exigence porte sur un préavis déclaré, pas sur une valeur; le centre seul peut arbitrer
SLO-05	fraction des soumissions acceptées en indisponibilité quantique totale	100 %	dérivation : propriété binaire de l'architecture; toute valeur inférieure signale un couplage non voulu entre l'accès et l'état des ressources

Tableau 16 — Cinq objectifs de service. **Deux sont en forme paramétrée assumée plutôt que chiffrés** : le travail préfère un objectif dont le paramètre est déclaré à un chiffre inventé.

La règle qui gouverne l'établissement d'un objectif de service est explicite et restrictive : la justification doit relever de l'une de trois formes — donnée sourcée, dérivation explicite, forme paramétrée. **Si aucune des trois n'est possible, l'objectif n'est pas publié.** Un chiffre non fondé est pire qu'un objectif absent.

8.5 Les six procédures et ce qu'elles propagent

Procédure	Objet	Ce qu'elle propage
PR-01	Fenêtre d'étalonnage planifiée	allonge la file (B2), déplace des charges (B4), consomme le quota d'autres tenants (B3)
PR-02	Revue périodique de la couverture de télémétrie	retirer une métrique supprime la possibilité de constater un effet systémique; l'étape de retrait ne se fait pas sans relire les quatre boucles
PR-03	Établissement et révision d'un objectif de service	un objectif publié devient un seuil actif dans d'autres procédures; le réviser modifie leur comportement, pas seulement la promesse
PR-04	Arbitrage de contention et application du plancher d'équité	le plancher est compensatoire, donc en retard d'une fenêtre de mesure; son mécanisme est spécifié à la suite de cette table
PR-05	Détection de dérive, requalification, retrait	le retrait déplace toutes les charges vers les ressources restantes (B4), donc accélère l'atteinte des quotas (B3)
PR-06	Perte de contact avec une ressource	même propagation que PR-05, sans le préavis qui la rendait planifiable

Tableau 17 — Six procédures d'exploitation. Chacune porte six champs dans le mémoire : déclencheur, acteur, étapes, critère d'arrêt, point de reprise, et **fondement** — ce qui la soutient dans les sources, ou la déclaration qu'elle n'est soutenue par aucune.

Le plancher d'équité : mécanisme et retard. La première version déclarait le plancher compensatoire sans en spécifier la forme; la révision la donne. Lorsque `part_serve_tenant` (§ 8.3) reste sous le plancher déclaré pendant une fenêtre de mesure complète, PR-04 exempte les charges du tenant lésé de la règle R4 et les sert en tête de file d'admission, jusqu'au retour au plancher — un surclassement déclenché et borné par la même télémétrie qui le mesure, sans paramètre nouveau. Ce que le mécanisme ne répare pas est nommé : le retard d'une fenêtre de mesure demeure — un plancher proactif exigerait un contrôle d'admission prédictif, hors de portée de la méthode (§ 3.3) —, et la réserve du § 9.1 est maintenue pour cette raison.

La première étape de PR-05 traite le point de fragilité assumé au § 6.4 : l'état `degrade` confond une dégradation physique et une péremption de la connaissance, et l'information distinctive n'existe que dans le champ `cause`. La procédure commence donc par lire ce champ et par bifurquer — si l'origine est la péremption, la réponse est un étalonnage, pas un incident.

Les six procédures ont été confrontées à la table de transitions case par case sur les transitions qu'elles invoquent. Aucune ne suppose une transition absente de la table, et aucune ne contredit une case « sans effet ». En particulier, PR-06 respecte la règle selon laquelle le retour de service passe toujours par `etalonnage`.

Une propriété transversale des six mérite d'être nommée depuis la troisième strate (§ 2.8) : chaque procédure porte un déclencheur, un critère d'arrêt et un point de reprise idempotent, et aucune ne tient d'état dans la tête de l'opérateur. Elles sont donc spécifiées sous la forme qu'un opérateur automatisé peut exécuter — celle que la littérature agentique réclame quand elle nomme l'auditabilité et la reproductibilité comme verrous [71]. C'est une propriété constatable par lecture, non une revendication : aucun agent ne les a exécutées, et rien ici ne mesure ce qu'une exécution automatisée donnerait.

8.6 L'intégrité de l'entrée d'étalonnage

La projection consomme une entrée que la plateforme ne produit pas : les propriétés et valeurs de fidélité déclarées par le dispositif ou par son fournisseur (interface I8). INV-5 traite l'information **absente**; il ne

dit rien de l'information **fausse**, et la première strate ne documentait pas cette menace. La seconde la chiffre : d'après son résumé, une étude mesure qu'un adversaire logé dans la chaîne d'étalonnage, en déclarant des taux d'erreur faussés sans toucher au matériel, dégrade l'allocation de deux cadres publiés — latence d'exécution accrue de 24 %, probabilité de succès réduite de 7,8 % [66] — et la multi-tenance ouvre des canaux propres, dont l'injection adverse de portes [67].

La réponse retenue est minimale et observable. La cohérence statistique des données d'étalonnage d'un cycle à l'autre est surveillée — la piste de défense que la source elle-même propose [66] —, la ligne `coherence_etalonnage` du § 8.3 porte cette surveillance, et un écart au-delà du seuil déclaré déclenche PR-05, dont la première étape lit déjà la cause de la transition avant de bifurquer. Aucune exigence nouvelle n'est ajoutée : le mécanisme s'appuie sur des éléments existants — télémétrie, procédure, machine d'états — et c'est ce qui le rend spécifiable sans matériel.

Ce que cette réponse ne couvre pas est déclaré plutôt que masqué : un adversaire qui fausse les données de manière **statistiquement cohérente** d'un cycle à l'autre reste indétectable par ce mécanisme, et la vérification indépendante de l'étalonnage — refaire la mesure plutôt que la croire — relève du banc d'essai [51], hors de la frontière du système du § 5.1.

Le septième archétype (§ 5.2) monte l'enjeu d'un cran sans changer la réponse. Quand le consommateur de l'état est un agent qui décide seul, une entrée d'étalonnage faussée n'égare plus un opérateur : elle s'injecte dans la boucle de décision d'un système autonome, sans le regard humain qui pourrait s'arrêter sur une valeur invraisemblable. Le mécanisme reste le même — cohérence surveillée, PR-05 —, mais c'est pour ce consommateur-là qu'il cesse d'être optionnel, et la littérature des laboratoires autonomes nomme précisément l'auditabilité parmi ses verrous centraux [71].

9 Validation

9.1 Matrice de couverture

Une ligne par exigence, sans exception. Une exigence non couverte reste dans la matrice avec son verdict; l'en retirer pour verdir le tableau serait le défaut que la méthode interdit explicitement.

Verdict	Effectif	Sens
Couverte	14	un élément d'architecture nommé la réalise et le moyen de vérification a été appliqué
Couverte sous réserve	4	réalisée, avec une limite déclarée qui survit à l'architecture : plancher d'équité compensatoire, modèle tarifaire laissé au centre, portabilité sans mécanisme spécifié, isolation des effets observables du partage
Conditionnelle	2	réalisée sous la topologie nominale T2, pas sous la variante T1 : réservation des fenêtres non réalisable, préavis dégradé
Non couverte	0	—

Tableau 18 — Répartition des verdicts sur les vingt exigences.

Ce résultat mérite d'être regardé avec méfiance plutôt qu'avec satisfaction. Les vingt exigences ont été écrites par le même auteur que l'architecture qui les réalise, dans un ordre où les exigences précèdent l'architecture. Aucune n'a été formulée par un tiers, et aucune ne provient d'un besoin exprimé plutôt qu'inféré. Un taux de couverture de cent pour cent dans ces conditions mesure la cohérence interne du travail, non son adéquation à un besoin réel. Le § 10.1 en fait une menace à part entière.

Les quatre réserves sont instructives parce qu'aucune n'est technique au sens strict. Deux tiennent à un modal *devrait* dans l'énoncé de l'exigence — la satisfaire en tant que déclaration n'est pas la satisfaire en tant que règle. Une tient à un mécanisme correct mais en retard. Une tient à une contradiction interne assumée : l'isolation du contenu entre tenants est assurée, mais celle des effets observables du partage ne l'est pas et ne peut pas l'être sans contredire l'exigence d'observabilité de la décision.

9.2 Rejeu des scénarios

Scénario	Verdict	Ce qui manque
Soumission hybride	satisfait	—
Campagne variationnelle	satisfait	aucun seuil chiffré d'inactivité classique acceptable
Étalonnage planifié	satisfait	—
Dégradation imprévue	partiellement satisfait	le point de contrôle de campagne (§ 7.4), spécifié en révision, rouvre la décision d'une charge itérative sur événement d'état; un noyau monolithique reste sans réévaluation possible en cours d'exécution — la chronologie de la Fig. 6 montre exactement où
Multi-tenance sous contention	partiellement satisfait	le plancher d'équité est désormais spécifié (§ 8.5) mais demeure compensatoire, donc en retard d'une fenêtre; l'exemption de plafond de la classe <code>precision</code> (règle R4) reste une fenêtre d'iniquité assumée, que le surclassement de PR-04 compense sans la fermer
Campagne pilotée par agent	satisfait	le rejeu ne mobilise que des mécanismes existants — soumission unifiée, décision par campagne (§ 7.5), point de contrôle sur événement d'état (§ 7.4), borne R4 et plancher PR-04; ce qui manque est une mesure : l'ampleur de la résonance du § 6.5 est une hypothèse déclarée, que <code>taux_report</code> et <code>profondeur_file</code> rendraient observable

Tableau 19 — Rejeu des six scénarios opérationnels. Les deux résidus sont laissés visibles plutôt que réparés à la hâte; la révision en a réduit la surface — point de contrôle de campagne, mécanisme de plancher — sans les faire disparaître.

Les deux résidus sont liés à des choix explicites du § 7, et c'est ce qui les rend utiles. Le premier découle de ce que la décision est prise une fois, avant l'exécution — conséquence directe d'INV-4, qui interdit les signaux post-exécution : le point de contrôle de campagne en réduit la portée aux seuls noyaux monolithiques, il ne l'abolit pas. Le second découle de la règle R4, dont le § 7.2 avait annoncé qu'elle tranchait en faveur de l'utilité contre l'équité. Une spécification dont les défauts se déduisent de ses décisions publiées est réparable; une spécification dont les défauts surprennent son auteur ne l'est pas.

9.3 Confrontation à trois plateformes

Deux questions par plateforme : l'architecture proposée **expliquerait-elle** ce que la plateforme fait, et **l'améliorerait-elle**?

LUMI-Helmi, couplage de type service [17]. Elle l'expliquerait en partie — et avec un verdict peu flatteur qu'elle porte elle-même : sous cette topologie T1, deux de ses propres exigences deviennent non réalisable et dégradée. L'apport identifiable serait la projection, mais la matière d'entrée se limite à ce que le fournisseur publie. L'amélioration est donc conditionnelle à une information que la plateforme ne contrôle pas, et aucune donnée publiée ne permet de savoir si elle serait significative.

QMIO, intégration serrée avec déchargement de noyaux [16]. Elle l'expliquerait dans sa structure : c'est la topologie T2, voire T3 pour la partie déchargement. L'améliorerait-elle? **Impossible à établir** : les versions consultées ne publient pas les mesures d'exploitation qui permettraient la comparaison. C'est le verrou V6 rencontré au point où il fait le plus mal — la plateforme qui documente le mieux son cycle complet ne publie pas ce qu'il faudrait pour juger une proposition d'amélioration, et la réponse « on ne peut pas savoir » vaut pour toute proposition, pas seulement pour celle-ci.

SuperMUC-NG avec MQSS et un QPU de vingt qubits [18], [19], [68]. C'est le meilleur ajustement des trois : les QPU y sont exposés en ressources génériques sans modification du cœur de l'ordonnanceur, avec un ordonnancement à deux niveaux [19], et le réétalonnage y est décrit comme automatique et pilotable par l'ordonnanceur HPC [68] — le seul cas recensé où le principe d'INV-3 est effectivement appliqué en production. L'apport serait la projection et une politique qui consomme la fidélité comme critère de placement; rien dans les sources consultées n'indique que la politique d'ordonnancement de ce déploiement l'utilise, mais *c'est une inférence de l'auteur, non un constat*. Si la fidélité y est déjà consommée sans que les sources le disent, l'apport se réduit à la formalisation.

Sur la portabilité, cette plateforme fait mieux que l’architecture proposée : le MQSS porte un compilateur fondé sur MLIR et des adaptateurs pour les cadriciels répandus [18], là où la fonction d’adaptation proposée ici s’arrête au *backend* sans en spécifier le mécanisme. C’est la conséquence directe et prévue de la décision du § 6.3.

10 Menaces à la validité et conditions de réfutation

10.1 Dix menaces

Menace Énoncé	
M1	Biais de sources. Le texte normatif de référence n’a pas été acquis, et une part importante de la base est constituée de préimpressions.
M2	Obsolescence du domaine. Plusieurs sources primaires ont moins de dix-huit mois; le domaine bouge en mois, pas en années.
M3	Absence d’implémentation de la plateforme. La plateforme n’existe pas et aucune charge réelle n’a été exécutée. La politique, elle, dispose depuis la révision d’une implémentation de référence qui rejoue les déroulés du § 7.5 et vérifie la table de transitions — un rejeu de valeurs spécifiées, non une mesure.
M4	Généralisation depuis peu de plateformes. Six plateformes recensées, trois confrontées.
M5	Parties prenantes archétypales. Sept archétypes inférés de traces écrites, zéro entrevue.
M6	Absence de contradicteur externe. Aucune revue adverse avant publication.
M7	Biais de confirmation d’un auteur unique. Les décisions et leur évaluation partagent un seul jugement.
M8	Le juge et la partie. Les trois rôles du dispositif de qualité — fixer le contrat, l’exécuter, juger la conformité — sont tenus par la même personne.
M9	Ancrage asymétrique résiduel du corpus. Vingt-huit des trente-trois sources de seconde strate restent ancrées sur leur seul résumé. Les cinq qui portaient la revendication centrale — les quatre quasi-antériorités du § 2.6 et [31] — ont été relues en texte intégral en révision.
M10	Strate de motivation jeune et contestée. La troisième strate (§ 2.8) est entièrement ancrée sur résumé; une de ses sources centrales a été corrigée par sa revue après contestation publique de ses revendications [26], [69], et une autre est une préimpression [27]. Aucune affirmation de performance n’en est tirée, et son usage est confiné à la motivation, un archétype, un scénario et un effet émergent — si la strate tombait entière, aucune contribution des § 6 à 8 ne tomberait avec elle.

Tableau 20 — Dix menaces à la validité, documentées plutôt que mentionnées.

Quatre pèsent plus lourd que les autres et méritent d’être développées.

M8 — le juge et la partie. L’indice est publié : onze spécifications conformes sur onze, dix-huit exigences couvertes sur vingt, aucun critère jugé inatteignable. **Un tel taux, dans un travail de cette ampleur, s’explique plus vraisemblablement par une calibration des critères sur ce qui allait être produit que par une exécution sans faute.**

M9 — l’ancrage asymétrique, et ce que la révision en a fait. La revendication centrale avait été resserrée (§ 2.6, § 6.3) sur la foi de résumés, et la première version désignait la lecture intégrale des quatre quasi-antériorités comme le premier test de la contribution. Ce test a été conduit en révision : aucune des quatre ne publie un état daté consommé par un gestionnaire de charge, et RÉF-2 n’est pas déclenchée (§ 10.2). La menace ne disparaît pas pour autant : vingt-huit sources restent ancrées sur résumé, le texte intégral de [64] n’a pas pu être acquis, et une antériorité peut se loger hors du corpus — dans le canon distribué classique que la révision n’a balayé que par ses deux spécifications les plus connues (§ 2.4), ou ailleurs.

Aucune calibration numérique. La politique du § 7 est une structure sans valeurs fondées : quinze poids, un seuil de confiance, une borne de report et deux bornes de régime sont des hypothèses de travail. L’écart entre livrer une structure et livrer un système réglé est plus grand que le mot « spécification » ne le suggère.

M3, et ce qu’il ne menace pas. L’absence d’implémentation de la plateforme invalide toute affirmation de performance, et l’article n’en fait aucune. Elle n’invalide pas les propriétés vérifiables par lecture : la totalité de la machine d’états, le déterminisme de la politique, la complétude de la table des entrées —

propriétés que le script de rejeu versionné vérifie désormais aussi par exécution. C’est la distinction que le § 10.2 exploite pour rendre deux conditions de réfutation exécutables sans matériel.

10.2 Huit conditions de réfutation

Chaque condition décrit un fait constatable par un tiers. Une condition inobservable — « si l’approche est mauvaise » — ne compte pas. Le préfixe RÉF les distingue des règles de filtrage R1 à R6 de la politique (§ 7.2), avec lesquelles elles n’ont aucun rapport.

RÉF-1 Une plateforme HPC-QPU exploite déjà une politique de délégation consommant la fidélité comme critère de placement, documentée publiquement. **Effet** : V2 et V3 se referment, et la contribution du § 7 se réduit à une formalisation de l’existant.

RÉF-2 Une interface existante porte déjà l’état ordonnable — une version publiée de QRMI, de QDMI ou d’un équivalent exposant un état à plus de deux valeurs, consommable par un gestionnaire de charge sans projection intermédiaire. **Effet** : la contribution architecturale centrale du § 6.3 disparaît, et V2 était mal caractérisé. **Premier test conduit en révision** : la lecture intégrale du candidat le plus proche — l’instantané contractuel de [43] — confirme un contrat immuable lié à la réclamation, sans machine d’états ni consommation par une politique de placement, et l’objet virtualisé y est un simulateur. La condition n’est pas déclenchée; elle reste ouverte pour toute interface non recensée — y compris une réalisation par ClassAds [39] qui porterait une fidélité datée à péremption événementielle, qu’aucune source consultée ne publie.

RÉF-3 L’étalonnage cesse d’occuper la ressource — une publication établissant qu’un réétalonnage s’exécute pendant qu’une charge utilisateur occupe le dispositif, avec mesure de l’effet sur la charge. **Effet** : INV-3 perd sa portée, la fenêtre d’étalonnage cesse d’être un objet d’ordonnancement, et une part importante des § 6 et 8 tombe. C’est la condition la plus large; la cadence de réétalonnage à l’échelle de la milliseconde [2] la rend plausible sans l’établir.

RÉF-4 La dérive de fidélité est indépendante de l’intensité d’usage — une mesure corrélant `pente_derive` et `profondeur_file` ne montrant aucune dépendance. **Effet** : la boucle B2 n’existe pas, le risque de synchronisation des fenêtres disparaît, et l’argument de disponibilité en faveur d’une flotte hétérogène tombe.

RÉF-5 Une des sept exigences du § 8.1 se retrouve à énoncé équivalent dans la documentation d’exploitation d’une plateforme HPC — ou d’une infrastructure de grille — sans QPU. **Effet** : Q3 reçoit une réponse négative pour l’exigence concernée; si les sept y figurent, la question entière tombe. La révision a élargi la condition à la grille (§ 2.4) : c’est là que le candidat le plus sérieux se logerait.

RÉF-6 La politique du § 7.2 produit, pour les entrées du § 7.5, une décision différente de celle qui y est calculée. **Effet** : la spécification n’est pas déterministe, ou le déroulé est faux; dans les deux cas la propriété centrale revendiquée au § 7 est en défaut. Le script de rejeu versionné exécute ce calcul; réfuter la condition demande d’exhiber une trace divergente, du script ou d’un calcul manuel.

RÉF-7 La fraîcheur de la métrique de fidélité est sans effet sur la qualité du placement — une mesure montrant que des décisions calculées sur une métrique périmée depuis plusieurs cycles d’étalonnage obtiennent des résultats indistinguables de décisions prises sur métrique fraîche. **Effet** : l’événement E9, la règle R1b et la ligne `age_metrrique_fidelite` perdent leur objet, et la projection se réduit à un instantané statique par ressource. Des résultats de compilation la rendent plausible sans l’établir : des circuits compilés une fois se réutilisent sur plusieurs cycles d’étalonnage sans perte significative [31] — mais réutiliser un circuit compilé n’est pas décider d’un placement, et la même étude mesure qu’une compilation sensible au bruit concentre la charge et accroît la variabilité des sorties. Sa lecture intégrale, conduite en révision, borne ce qu’elle peut établir : seize algorithmes, six machines de 127 qubits, un mois — une mesure de réutilisation de transpilation **sur une même machine**, qui ne traite ni du choix entre ressources ni de la fraîcheur comme critère de sélection. La condition reste ouverte; cette source-là ne peut pas la déclencher.

RÉF-8 Un laboratoire autonome publié opère un instrument dont la qualité de sortie dérive et se périmé — QPU compris — sans état de ressource daté ni péremption événementielle, en re-mesurant la qualité au sein de chaque itération de sa boucle, avec des résultats de campagne équivalents. **Effet** : la motivation par la science autonome (§ 1, § 2.8) s'affaiblit — un agent peut alors se passer de la projection en payant la re-mesure —, et le septième archétype (§ 5.2) perd son besoin distinctif; la spécification des § 6 à 8 demeure, mais son consommateur le plus exigeant lui est retiré. La parenté avec RÉF-7 est déclarée : RÉF-7 attaque la valeur de la **fraîcheur**, RÉF-8 celle de sa **publication** — la re-mesure privée remplaçant l'état publié. Le cadre d'exécution conditionnelle de [47], qui évalue les ressources côté application, est la forme embryonnaire de ce que cette condition décrirait à l'échelle d'un laboratoire entier.

RÉF-5 et RÉF-6 sont exécutables immédiatement, sans matériel : l'une demande une lecture comparée, l'autre un calcul. Ce sont les deux que cet article invite le plus directement un lecteur à tenter.

11 Conclusion

Le problème n'était pas un manque de briques, mais un manque de chaîne. Les architectures de référence existent, les interfaces exposent le processeur quantique comme ressource ordonnançable, l'intergiciel adaptatif fonctionne, les stratégies de partage sont mesurées et les plateformes tournent. Ce qui manquait se situe entre ces briques : une fonction de décision qui consomme l'état de la ressource, et une discipline d'exploitation qui traite l'instabilité de cette ressource comme un état normal plutôt que comme une panne.

Quatre éléments sont proposés pour combler cet intervalle. La projection de l'état de ressource transforme les propriétés physiques changeantes d'un dispositif en un état daté que l'ordonnanceur consomme — c'est la pièce que les interfaces existantes ne couvrent pas et que la littérature ne réalise que par fragments (§ 2.6), et elle vient avec la machine d'états totale qui la porte et l'analyse des boucles qu'elle ferme. La politique de délégation est déterministe et totale, et un lecteur peut refaire le calcul à la main. La règle de sélection entre stratégies de partage comble un vide que l'état de l'art avait laissé explicite. Les sept exigences d'exploitabilité, six procédures, vingt métriques et cinq objectifs de service produisent ce que la littérature opérationnelle ne fournit pas, en déclarant à chaque fois ce qui les soutient.

Ces éléments portent leurs limites avec eux. La plus lourde n'est pas technique : le dispositif de qualité a été tenu par une seule personne, et un taux de couverture élevé mesure alors la cohérence interne d'un travail, non son adéquation à un besoin réel. La politique est structurellement spécifiée et numériquement non calibrée. Deux scénarios sur cinq ne passent que partiellement, et leurs défauts sont laissés visibles — l'un parce qu'INV-4 interdit les signaux post-exécution, l'autre parce que la règle R4 tranche en faveur de l'utilité contre l'équité. Ce sont des conséquences de décisions publiées, pas des surprises.

La recherche complémentaire conduite en fin de travail, puis la révision, ont fait subir à cette discipline ses deux premiers tests. Trente-trois sources ajoutées en seconde strate ont produit quatre quasi-antériorités, une contestation de la prémisse de fraîcheur convertie en condition de réfutation (RÉF-7), et une menace que la première strate n'avait pas vue — l'intégrité de l'entrée d'étalonnage (§ 8.6). La révision a ensuite conduit ce que la première version différait : la lecture intégrale des quatre quasi-antériorités et de la source de RÉF-7 — aucune ne déclenche sa condition, une valeur corrigée en sort —, l'extension de la revue au canon distribué classique, qui a déplacé la revendication du mécanisme vers le contenu (§ 2.4), la spécification des deux mécanismes que le jeu des scénarios avait montrés manquants (point de contrôle de campagne, plancher d'équité), et une implémentation de référence de la politique. La revendication centrale en sort resserrée, repositionnée et testée plutôt qu'affaiblie; l'asymétrie d'ancrage résiduelle demeure une menace déclarée (M9).

La seconde révision situe enfin l'exploitation. Une troisième strate courte — laboratoires autopilotés, agents scientifiques autonomes (§ 2.8) — établit le consommateur pour lequel la chaîne de délégation cesse d'être un confort : un agent qui prend le centre pour instrument n'a que l'état publié, la décision motivée et les procédures reconstituables pour opérer, et c'est précisément ce que cette architecture spécifie. L'article lui

répond par un archétype (§ 5.2), un scénario (§ 5.3), un effet émergent (§ 6.5) et une condition de réfutation (RÉF-8), sans modifier aucune contribution — et en portant la jeunesse de cette littérature comme menace (M10) plutôt que comme promesse.

Ce que l'article livre est donc un objet précis : une architecture et une spécification dont chaque affirmation porte sa source ou son statut d'hypothèse, dont chaque décision porte ses options écartées, et dont chaque faiblesse porte la condition qui la révélerait. C'est ce qu'un travail mené sans contradictoire peut offrir de mieux : non pas la certitude d'avoir raison, mais les moyens précis de démontrer qu'il a tort.

Disponibilité. Cet article est tiré d'un mémoire technologique dont les spécifications de chapitre, les registres d'exigences et de décisions, les journaux d'ancrage et la chaîne d'assemblage sont versionnés; ces artefacts sont disponibles auprès de l'auteur. L'implémentation de référence de la politique — `rejeu-politique.py`, qui reproduit les déroulés du § 7.5, l'analyse de sensibilité et la vérification de la table de transitions — est versionnée aux côtés des sources de l'article. Les huit figures et l'ensemble des tableaux sont produits par l'auteur; aucune figure n'est reprise d'une source.

Références

- [1] K. Yeter-Aydeniz *et al.*, « Measuring NISQ Gate-Based Qubit Stability Using a 1+1 Field Theory and Cycle Benchmarking ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2201.02899>
- [2] M. A. Marciniak et others, « Millisecond-Scale Calibration and Benchmarking of Superconducting Qubits ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2602.11912>
- [3] S. Seelam *et al.*, « Reference Architecture of a Quantum-Centric Supercomputer ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2603.10970>
- [4] A. Shehata *et al.*, « Quantum-HPC Software Stacks and the openQSE Reference Architecture: A Survey ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2604.20912>
- [5] S. Raj, S. Sai, Y. Simmhan, K. Chard, et R. Buyya, « Quantum Integrated High-Performance Computing: Foundations, Architectural Elements and Future Directions ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2604.19814>
- [6] A. Shehata *et al.*, « Bridging Paradigms: Designing for HPC-Quantum Convergence », *Future Generation Computer Systems*, vol. 174, 2026, [En ligne]. Disponible sur: <https://arxiv.org/abs/2503.01787>
- [7] Oak Ridge National Laboratory / U.S. DOE, « Integrating Quantum Computing Resources into Scientific HPC Ecosystems ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2408.16159>
- [8] Munich Quantum Software Stack, « QDMI — Quantum Device Management Interface ». [En ligne]. Disponible sur: <https://github.com/Munich-Quantum-Software-Stack/QDMI>
- [9] T. Badts *et al.*, « Examining QRMI as a Unified Interface for Quantum-HPC Integration ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2607.19591>
- [10] Qiskit Community / IBM Quantum, « Quantum Resource Management Interface (QRMI) ». [En ligne]. Disponible sur: <https://github.com/qiskit-community/qrmi>
- [11] U. Bacher *et al.*, « Quantum Resources in Resource Management Systems ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2506.10052>
- [12] P. Mantha, F. J. Kiwit, N. Saurabh, S. Jha, et A. Luckow, « Hybrid Quantum-HPC Middleware Systems for Adaptive Resource, Workload and Task Management ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2604.03445>
- [13] M. Cipollini *et al.*, « Three Ways to Share a QPU: Scheduling Strategies for Hybrid Quantum-HPC Applications ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2604.14955>
- [14] P. Das et others, « A Case for Multi-Programming Quantum Computers », dans *Proc. MICRO-52*, 2019. [En ligne]. Disponible sur: <https://doi.org/10.1145/3352460.3358287>

- [15] S. Niu et A. Todri-Sanial, « Enabling Multi-Programming Mechanism for Quantum Computing in the NISQ Era », *Quantum*, vol. 7, p. 925, 2023, [En ligne]. Disponible sur: <https://doi.org/10.22331/q-2023-02-16-925>
- [16] J. Cacheiro *et al.*, « QMIO: A Tightly Integrated Hybrid HPC-QC System ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2505.19267>
- [17] CSC — IT Center for Science, « Finland Opens Quantum Computer for Research Purposes ». [En ligne]. Disponible sur: <https://csc.fi/en/news/finland-opens-quantum-computer-for-research-purposes-the-fusion-of-quantum-computing-and-supercomputing-enables-completely-new-science/>
- [18] L. Burgholzer *et al.*, « The Munich Quantum Software Stack: Connecting End Users, Integrating Diverse Quantum Technologies, Accelerating HPC ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2509.02674>
- [19] M. N. Farooqi *et al.*, « Enabling Hybrid HPCQC Workflows with a Heterogeneous Software Stack ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2608.14827>
- [20] M. Slys *et al.*, « Hybrid Classical-Quantum Supercomputing: A Demonstration of a Multi-user, Multi-QPU and Multi-GPU Environment ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2508.16297>
- [21] Forschungszentrum Jülich, « JUNIQ — Jülich UNified Infrastructure for Quantum Computing ». [En ligne]. Disponible sur: <https://www.fz-juelich.de/en/research/research-fields/research-infrastructures/juniqu>
- [22] EuroHPC Joint Undertaking, « Our Quantum Computers ». [En ligne]. Disponible sur: https://www.eurohpc-ju.europa.eu/eurohpc-quantum-computers/our-quantum-computers_en
- [23] K. A. Britt et T. S. Humble, « High-Performance Computing with Quantum Processing Units ». [En ligne]. Disponible sur: <https://arxiv.org/abs/1511.04386>
- [24] M. Abolhasani et E. Kumacheva, « The Rise of Self-Driving Labs in Chemical and Materials Sciences », *Nature Synthesis*, vol. 2, p. 483-492, 2023, [En ligne]. Disponible sur: <https://doi.org/10.1038/s44160-022-00231-0>
- [25] D. A. Boiko, R. MacKnight, B. Kline, et G. Gomes, « Autonomous Chemical Research with Large Language Models », *Nature*, vol. 624, p. 570-578, 2023, [En ligne]. Disponible sur: <https://doi.org/10.1038/s41586-023-06792-0>
- [26] N. J. Szymanski et others, « An Autonomous Laboratory for the Accelerated Synthesis of Inorganic Materials », *Nature*, vol. 624, p. 86-91, 2023, [En ligne]. Disponible sur: <https://doi.org/10.1038/s41586-023-06734-w>
- [27] C. Lu, C. Lu, R. T. Lange, et others, « The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2408.06292>
- [28] I.-D. Gheorghe-Pop, N. Tcholtchev, T. Ritter, et M. Hauswirth, « Quantum DevOps: Towards Reliable and Applicable NISQ Quantum Computing », dans *Proc. IEEE Globecom Workshops 2020*, 2020. [En ligne]. Disponible sur: <https://doi.org/10.1109/GCWkshps50303.2020.9367411>
- [29] M. Beisel, J. Barzen, F. Leymann, L. Stiliadou, et B. Weder, « Observability for Quantum Workflows in Heterogeneous Multi-cloud Environments », dans *Proc. CAiSE 2024*, 2024, p. 612-627. [En ligne]. Disponible sur: https://doi.org/10.1007/978-3-031-61057-8_36
- [30] C. Cicconetti, « Modeling and Performance Evaluation of Hybrid Classical-Quantum Serverless Computing Platforms », *IEEE Transactions on Quantum Engineering*, vol. 6, p. 3101313, 2025, [En ligne]. Disponible sur: <https://ieeexplore.ieee.org/document/10989577/>
- [31] Y. Huo et others, « Revisiting Noise-adaptive Transpilation in Quantum Computing: How Much Impact Does it Have? ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2507.01195>

- [32] V. Sochat et D. Milroy, « Hybrid Quantum and Classical Workload Management with Graph-based Scheduling ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2607.09151>
- [33] A. Khalid *et al.*, « Impacts of Decoder Latency on a Utility-Scale Quantum Computer Architecture ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2511.10633>
- [34] N. Kanazawa, Y. Morohoshi, H. Takahashi, Y. Kawashima, H. Horii, et K. Nakajima, « Observability Architecture for Quantum-Centric Supercomputing Workflows ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2512.05484>
- [35] S. Chundury *et al.*, « Scaling Hybrid Quantum-HPC Applications with the Quantum Framework ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2509.14470>
- [36] STFC Hartree Centre, « Alice & Bob and STFC Hartree Centre Integrate Cat-Qubit Quantum Computers into Standard HPC Workflow and Job Scheduling Software (Slurm) ». [En ligne]. Disponible sur: <https://www.hartree.stfc.ac.uk/news/2025/11/06/alice-bob-and-stfc-hartree-centre-integrate-cat-qubit-quantum-computers-into-standard-hpc-workflow-and-job-scheduling-software-slurm/>
- [37] NVIDIA, « NVIDIA Introduces NVQLink — Connecting Quantum and GPU Computing ». [En ligne]. Disponible sur: <https://nvidianews.nvidia.com/news/nvidia-nvqlink-quantum-gpu-computing>
- [38] S. Caldwell et others, « Platform Architecture for Tight Coupling of High-Performance Computing with Quantum Processors ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2510.25213>
- [39] R. Raman, M. Livny, et M. Solomon, « Matchmaking: Distributed Resource Management for High Throughput Computing », dans *Proc. 7th IEEE Int. Symp. on High Performance Distributed Computing (HPDC-7)*, 1998. [En ligne]. Disponible sur: <https://doi.org/10.1109/HPDC.1998.709966>
- [40] S. Andreozzi *et al.*, « GLUE Specification v.\,2.0 », technical report GFD-R-P.147, 2009. [En ligne]. Disponible sur: <https://www.ogf.org/documents/GFD.147.pdf>
- [41] P. Viviani *et al.*, « Assessing the Elephant in the Room in Scheduling for Current Hybrid HPC-QC Clusters ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2504.10520>
- [42] N. Maring *et al.*, « One Nine Availability of a Photonic Quantum Computer on the Cloud toward HPC Integration ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2308.14582>
- [43] S. Liu et others, « HPC-vQPU: A Service-Export Architecture for Virtual QPUs on Batch-Scheduled HPC Systems ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2605.28845>
- [44] S. Pehlivanoglu et others, « Qurator: Scheduling Hybrid Quantum-Classical Workflows Across Heterogeneous Cloud Providers ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2604.05505>
- [45] P. D. Nation et M. Treinish, « Suppressing Quantum Circuit Errors Due to System Variability », *PRX Quantum*, vol. 4, p. 10327, 2023, [En ligne]. Disponible sur: <https://doi.org/10.1103/PRXQuantum.4.010327>
- [46] P. Murali, J. M. Baker, A. Javadi-Abhari, F. T. Chong, et M. Martonosi, « Noise-Adaptive Compiler Mappings for Noisy Intermediate-Scale Quantum Computers », dans *Proc. ASPLOS'19*, 2019. [En ligne]. Disponible sur: <https://doi.org/10.1145/3297858.3304075>
- [47] M. Lammers et others, « Quantum Resource Management in the NISQ Era: Challenges, Vision, and a Runtime Framework ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2508.19276>
- [48] H. Kurniawan et others, « On the Use of Calibration Data in Error-Aware Compilation Techniques for NISQ Devices ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2407.21462>
- [49] P. Senapati, S. Zhu, B. Peng, B. Fang, et Q. Guan, « UQ-VarQA: Benchmarking and Characterizing NISQ Computers Through Uncertainty Quantification of Variational Quantum Algorithms ». [En ligne]. Disponible sur: <https://ieeexplore.ieee.org/document/11311081/>
- [50] F. Berritta et others, « Efficient Qubit Calibration by Binary-Search Hamiltonian Tracking », *PRX Quantum*, vol. 6, p. 30335, 2025, [En ligne]. Disponible sur: <https://arxiv.org/abs/2501.05386>

- [51] T. Proctor, K. Young, A. D. Baczewski, et R. Blume-Kohout, « Benchmarking Quantum Computers », *Nature Reviews Physics*, vol. 7, p. 105-118, 2025, [En ligne]. Disponible sur: <https://arxiv.org/abs/2407.08828>
- [52] M. Bazayeva et others, « I-QMapper: Error-Aware Layout Optimization and Device Diagnostics for NISQ Hardware ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2606.27508>
- [53] L. Liu et X. Dou, « QuCloud: A New Qubit Mapping Mechanism for Multi-programming Quantum Computing in Cloud Environment », dans *Proc. IEEE HPCA 2021*, 2021, p. 167-178. [En ligne]. Disponible sur: <https://doi.org/10.1109/HPCA51647.2021.00024>
- [54] S. Upadhyay et S. Ghosh, « SHARE: Secure Hardware Allocation and Resource Efficiency in Quantum Systems ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2405.00863>
- [55] D. Bhounik, R. Majumdar, et S. Sur-Kolay, « Resource-Aware Scheduling of Multiple Quantum Circuits on a Hardware Device ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2407.08930>
- [56] R. Rocco et others, « Dynamic Solutions for Hybrid Quantum-HPC Resource Allocation ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2508.04217>
- [57] A. Esposito et others, « SLURM Heterogeneous Jobs for Hybrid Classical-Quantum Workflows ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2506.03846>
- [58] M. Tejedor, M. Grossi, C. Tüysüz, R. Rocha, et S. Vallecorsa, « Kubernetes-Orchestrated Hybrid Quantum-Classical Workflows ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2603.24206>
- [59] R. Zhou, Y. Gan, Y. Liu, et C. Qian, « CloudQC: A Network-aware Framework for Multi-tenant Distributed Quantum Computing », dans *Proc. IEEE ICDCS 2025*, 2025. [En ligne]. Disponible sur: <https://arxiv.org/abs/2504.20389>
- [60] S. Bahrani et others, « Resource Management and Circuit Scheduling for Distributed Quantum Computing Interconnect Networks », *IEEE Journal on Selected Areas in Communications*, vol. 44, p. 5175-5189, 2026, [En ligne]. Disponible sur: <https://arxiv.org/abs/2409.12675>
- [61] W. Luo et others, « A Simulation Framework for Workload Management in Hybrid Quantum-HPC Cloud System », dans *Proc. SC'25 Workshops*, 2025. [En ligne]. Disponible sur: <https://doi.org/10.1145/3731599.3767548>
- [62] N. Choudhury, A. S. Bhave, et K. Basu, « QUARTET: Quantum Utilization and Adaptation via Resource-Tuned Execution Techniques », dans *Proc. IEEE QCE 2025*, 2025. [En ligne]. Disponible sur: <https://ieeexplore.ieee.org/document/11250316/>
- [63] K. Rallis et others, « Interfacing Quantum Computing Systems with High-Performance Computing Systems: An Overview ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2509.06205>
- [64] J. A. Bravo-Montes et others, « Architectural Design and Orchestration of Heterogeneous Quantum-Classical Computing Systems ». [En ligne]. Disponible sur: <https://www.scitepress.org/PublishedPapers/2025/136536/>
- [65] R. Ramsauer et others, « Towards System-Level Quantum-Accelerator Integration ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2507.19212>
- [66] S. Das et others, « Impact of Error Rate Misreporting on Resource Allocation in Multi-tenant Quantum Computing and Defense », dans *Proc. GLSVLSI'25*, 2025. [En ligne]. Disponible sur: <https://doi.org/10.1145/3716368.3735191>
- [67] S. Upadhyay et S. Ghosh, « Stealthy SWAPS: Adversarial SWAP Injection in Multi-Tenant Quantum Computing », dans *Proc. VLSID 2024*, 2024. [En ligne]. Disponible sur: <https://doi.org/10.1109/VLSID60093.2024.00085>

- [68] E. Mansfield *et al.*, « First Practical Experiences Integrating Quantum Computers with HPC Resources: A Case Study With a 20-qubit Superconducting Quantum Computer ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2509.12949>
- [69] N. J. Szymanski et others, « Author Correction: An Autonomous Laboratory for the Accelerated Synthesis of Inorganic Materials », *Nature*, 2026, [En ligne]. Disponible sur: <https://doi.org/10.1038/s41586-025-09992-y>
- [70] A. V. Tobias et A. Wahab, « Autonomous 'Self-Driving' Laboratories: A Review of Technology and Policy Implications », *Royal Society Open Science*, vol. 12, p. 250646, 2025, [En ligne]. Disponible sur: <https://doi.org/10.1098/rsos.250646>
- [71] Z. Xie, M. Luo, Z. Ye, L. Chen, Z. Zhu, et J. Jiang, « Autonomous Chemistry and Materials Innovation Driven by Scientific Agents », *JACS Au*, vol. 6, p. 2659-2669, 2026, [En ligne]. Disponible sur: <https://doi.org/10.1021/jacsau.6c00213>
- [72] ISO/IEC/IEEE, « ISO/IEC/IEEE 15288:2023 — Systems and Software Engineering: System Life Cycle Processes. Texte normatif non acquis (barrière payante) : référent nommé pour identification, jamais mobilisé comme preuve ». 2023.
- [73] Wikipedia, « ISO/IEC 15288. Source tertiaire consultée le 2026-08-29, employée faute d'accès au texte normatif ». [En ligne]. Disponible sur: https://en.wikipedia.org/wiki/ISO/IEC_15288
- [74] P. Rao *et al.*, « A Performance Model for Hybrid Quantum-Classical Workflows ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2607.15426>
- [75] P. Döbler et M. S. Jattana, « A Survey on Integrating Quantum Computers into High Performance Computing Systems ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2507.03540>
- [76] E. Triantaphyllou, *Multi-criteria Decision Making Methods: A Comparative Study*, vol. 44. dans Applied Optimization, vol. 44. Springer, 2000. [En ligne]. Disponible sur: <https://doi.org/10.1007/978-1-4757-3157-6>
- [77] A. Wennersteen, M. Moreau, A. Nober, et M. Beji, « Towards a User-Centric HPC-QC Environment ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2509.20525>